

UNIVERSIDAD TECNOLÓGICA NACIONAL

Facultad Regional Rosario



Ingeniería en Sistemas de Información

REDES DE DATOS

Trabajo Práctico N.º 3

Capa de Transporte

Comisión

403

Integrantes

Juan Cruz Bonadeo — Legajo: 53533

Gonzalo Casermeiro — Legajo: 52674

Año 2026

Cátedra: Redes de Datos — Profesores: Cristian Medin / German Baró

Índice

1. Introducción.....	4
2. Fundamentos: la Capa de Transporte en los Modelos de Referencia.....	5
2.1. Ubicación y Función en los Modelos OSI y TCP/IP	5
2.2. Comunicación Extremo a Extremo frente a Salto a Salto	6
2.3. Evolución Histórica de los Protocolos de Transporte.....	7
3. Servicios y Primitivas de la Capa de Transporte	8
3.1. Servicio Orientado a Conexión y Servicio sin Conexión	8
3.2. La Entidad de Transporte, el Segmento y el Encapsulamiento	9
3.3. Primitivas del Servicio de Transporte.....	9
3.4. Sockets de Berkeley	10
3.5. Ejemplo de Interacción Cliente-Servidor.....	11
4. Elementos de los Protocolos de Transporte.....	12
4.1. Direccionamiento: TSAP y Puertos.....	12
4.2. Multiplexación y Demultiplexación	13
4.3. Establecimiento de la Conexión.....	14
4.4. Liberación de la Conexión	15
4.5. Control de Errores y Control de Flujo.....	16
4.6. Recuperación ante Caídas (Crash Recovery)	17
5. Control de Congestión en la Capa de Transporte	18
5.1. Asignación Deseable del Ancho de Banda	18
5.2. Regulación de la Tasa de Envío: AIMD	19
5.3. Congestión y Pérdidas en Redes Inalámbricas y Satelitales	20
6. Protocolo UDP (User Datagram Protocol).....	21
6.1. Características y Modelo de Servicio.....	21
6.2. Estructura del Encabezado UDP	22
6.3. Pseudoencabezado y Suma de Verificación.....	22
6.4. Aplicaciones sobre UDP: DNS, DHCP, RPC y RTP/RTCP	23
7. Protocolo TCP (Transmission Control Protocol).....	24
7.1. Modelo de Servicio de TCP	24
7.2. Estructura del Segmento TCP	25
7.3. Banderas de Control y Opciones	26
7.4. Establecimiento de la Conexión: Three-Way Handshake	27
7.5. Liberación de la Conexión	28

7.6. Máquina de Estados Finitos de TCP	29
7.7. Ventana Deslizante y Control de Flujo en TCP	29
7.8. Administración de Temporizadores.....	30
7.9. Control de Congestión en TCP	31
7.10. Variantes Modernas: CUBIC, Compound TCP y BBR	33
8. Comparativa TCP vs. UDP	34
9. Protocolos de Transporte Modernos	35
9.1. SCTP: Multihoming y Multistreaming	35
9.2. DCCP.....	36
9.3. QUIC y HTTP/3.....	36
9.4. MPTCP (Multipath TCP)	37
9.5. Comparativa de Protocolos de Transporte	38
10. Ejemplos Prácticos de Cálculo	38
10.1. Throughput Máximo Limitado por la Ventana de Recepción	39
10.2. Producto Ancho de Banda-Retardo y Escalado de Ventana	40
10.3. Evolución de la Ventana de Congestión	40
10.4. Cálculo del Temporizador de Retransmisión (RTO)	42
10.5. Eficiencia y Overhead de Encabezados	43
10.6. Agotamiento del Espacio de Números de Secuencia	44
11. Seguridad en la Capa de Transporte	45
11.1. Vulnerabilidades Históricas de TCP y UDP	45
11.2. Ataques de Denegación de Servicio: SYN Flood y Amplificación UDP ...	46
11.3. Escaneo de Puertos y Superficie de Ataque.....	46
11.4. TLS y DTLS: Seguridad sobre la Capa de Transporte.....	47
12. La Capa de Transporte en la Práctica Profesional	47
12.1. Impacto en el Desarrollo de Software	47
12.2. Herramientas de Diagnóstico: netstat, ss y Wireshark.....	48
12.3. NAT, Middleboxes y la Osificación del Transporte.....	48
12.4. Centros de Datos, Baja Latencia y Tendencias	49
12.5. Conectividad en Argentina y su Relación con el Transporte.....	49
13. Conclusión.....	49
14. Bibliografía	51
15. Anexo: Autoevaluación — Preguntas y Respuestas	53

1. Introducción

El presente trabajo práctico aborda la capa de transporte, el nivel de la arquitectura de red que transforma un servicio de entrega de paquetes entre máquinas — imperfecto, sin garantías y potencialmente desordenado— en un servicio de comunicación entre procesos que las aplicaciones pueden utilizar de manera directa. En el modelo OSI ocupa la capa 4 y, junto con la capa de red, constituye el «corazón» de la jerarquía de protocolos: por debajo de ella, la red lleva paquetes de un host a otro; por encima, las aplicaciones solo ven flujos de bytes o mensajes intercambiados con otro programa, sin conocer los routers ni las tecnologías que atraviesan.

La relevancia de esta capa se explica por una asimetría fundamental. La capa de red de Internet, basada en el Internet Protocol (IP), ofrece un servicio de «mejor esfuerzo»: los paquetes pueden perderse, duplicarse, llegar desordenados o sufrir retardos muy variables, y los routers pertenecen a operadores ajenos al usuario. La capa de transporte, en cambio, se ejecuta íntegramente en los hosts de los extremos, y es por lo tanto el primer nivel en el que el sistema final puede compensar las deficiencias de la red: allí se construye la confiabilidad, se regula la velocidad de envío y se distingue a qué proceso pertenece cada dato que llega a una máquina.

Los dos protocolos dominantes de Internet ilustran las dos filosofías de servicio posibles. El Transmission Control Protocol (TCP) es orientado a conexión y ofrece un flujo de bytes confiable y ordenado, con control de flujo y de congestión; sobre él funcionan la web tradicional, el correo y la transferencia de archivos. El User Datagram Protocol (UDP) es un protocolo sin conexión, con un encabezado de 8 bytes, que entrega datagramas independientes sin garantías y resulta preferible para la voz y el video en tiempo real o las consultas de DNS. Comprender por qué coexisten ambos es uno de los objetivos centrales de este informe.

La capa tampoco es un campo cerrado. TCP fue refinado durante cuatro décadas con nuevos algoritmos de control de congestión, y en los últimos años surgieron protocolos que cuestionan su hegemonía: el Stream Control Transmission Protocol (SCTP), Multipath TCP (MPTCP) y QUIC, estandarizado por la Internet Engineering Task Force (IETF) en 2021 como base de HTTP/3.

El trabajo recorre la capa desde sus fundamentos hasta su aplicación práctica: su ubicación en los modelos de referencia, sus servicios y primitivas —incluida la interfaz de sockets de Berkeley— y los elementos que todo protocolo de transporte debe resolver, desde el direccionamiento hasta el control de congestión. Luego se analizan en profundidad UDP y TCP, se los compara y se presentan SCTP, QUIC y MPTCP. Se completa con ejemplos de cálculo, consideraciones de seguridad, el papel del transporte en el desarrollo de software actual, una conclusión y un anexo de autoevaluación.

Para un ingeniero en sistemas, este conocimiento no es meramente teórico. Cada conexión a una base de datos, cada consumo de una API REST o cada WebSocket abierto utiliza un protocolo de transporte cuyas decisiones de diseño condicionan la latencia, el rendimiento y la robustez del sistema. Problemas cotidianos como el agotamiento de puertos efímeros, una transferencia que no aprovecha un enlace por una ventana demasiado pequeña o una operación que se ejecuta dos veces por un reintento mal diseñado se explican con los conceptos que se desarrollan a lo largo de este trabajo.

2. Fundamentos: la Capa de Transporte en los Modelos de Referencia

Esta sección precisa qué lugar ocupa la capa de transporte en las arquitecturas de red, qué la diferencia de las capas inferiores y cómo evolucionaron los protocolos que la implementan.

2.1. Ubicación y Función en los Modelos OSI y TCP/IP

En el modelo Open Systems Interconnection (OSI) de la International Organization for Standardization (ISO), la capa de transporte es la cuarta de siete. Recibe servicios de la capa de red, que entrega paquetes entre hosts, y ofrece servicios a la capa de sesión. Su objetivo, según la bibliografía de la cátedra, es brindar a las capas superiores un servicio de transmisión de datos confiable, eficiente y costo-efectivo, independiente de las redes subyacentes. Actúa como frontera entre las capas orientadas a la red, que dependen del operador, y las orientadas a la aplicación, que solo dependen del software de los usuarios.

En el modelo TCP/IP, el que efectivamente se implementa en Internet, la capa de transporte se ubica sobre la capa de internet y debajo de la de aplicación, que absorbe las funciones de sesión y presentación de OSI. Por eso tareas como el cifrado quedan a cargo de bibliotecas como Transport Layer Security (TLS), que pese a su nombre se ubica por encima de TCP. La siguiente tabla compara ambos modelos:

Aspecto	Modelo OSI	Modelo TCP/IP
Posición	Capa 4 de 7	Sobre internet (IP)
Capa superior	Sesión	Aplicación
Unidad de datos	TPDU / segmento	Segmento o datagrama
Protocolos	TP0 a TP4 (ISO 8073)	TCP, UDP, SCTP, QUIC

Aun con nomenclaturas distintas, ambos modelos asignan a la capa el mismo rol: abstraer la red y ofrecer comunicación entre procesos. Las cinco clases de protocolos OSI tuvieron escasa adopción, mientras que TCP y UDP se convirtieron en el estándar de facto.

Conviene distinguir servicio de protocolo. El servicio es lo que la capa ofrece hacia arriba —por ejemplo, «un flujo de bytes confiable»— y se expresa mediante primitivas; el protocolo es el conjunto de reglas y formatos que dos entidades pares intercambian para materializarlo. Esta separación permitió reemplazar decenas de veces el control de congestión de TCP sin modificar una sola línea de las aplicaciones.

2.2. Comunicación Extremo a Extremo frente a Salto a Salto

La capa de transporte es la primera que opera «de extremo a extremo». Las capas física, de enlace y de red trabajan «salto a salto»: cada protocolo de enlace dialoga solo con el dispositivo vecino, y cada router decide el siguiente salto. En cambio, el encabezado de transporte lo genera el host de origen y solo lo interpreta el de destino; los routers lo transportan como carga útil, sin importar cuántas redes existan en el camino.

De allí surge el argumento extremo a extremo de Saltzer, Reed y Clark (1984): ciertas funciones solo pueden implementarse correctamente con la participación de

los extremos, y replicarlas en la red es, a lo sumo, una optimización. La confiabilidad es el ejemplo clásico. Aunque cada enlace verifique sus tramas, un paquete puede corromperse en la memoria de un router o descartarse por desbordamiento de una cola, y ninguna de esas fallas es visible para la capa de enlace. Solo un control extremo a extremo —que detecta el hueco en los números de secuencia y solicita la retransmisión— garantiza que los datos recibidos son los enviados.

Este enfoque explica también por qué la capa reside en los hosts: así como en capa 3 los routers son los elementos principales, en capa 4 lo son los hosts. El núcleo de la red permanece simple, sin estado por conexión, y la innovación solo requiere actualizar los extremos. La realidad matiza este principio: firewalls con estado, traductores de direcciones (NAT) y balanceadores inspeccionan y modifican los encabezados de transporte, fenómeno conocido como osificación, que motivó la decisión de construir QUIC sobre UDP.

2.3. Evolución Histórica de los Protocolos de Transporte

A comienzos de los años setenta, los hosts de ARPANET se comunicaban mediante el Network Control Program (NCP), que asumía una subred confiable. Al interconectarse ARPANET con redes de radio y satelitales menos confiables, se necesitó un protocolo que garantizara la entrega sobre redes heterogéneas. En 1974 Cerf y Kahn publicaron el diseño del Transmission Control Program, que en 1978 se dividió en IP y TCP; la migración definitiva de ARPANET a TCP/IP ocurrió el 1 de enero de 1983. La siguiente tabla resume los hitos principales:

Protocolo	Año	RFC	Orientación	Característica principal
NCP	1970	RFC 33	Con conexión	Suponía una subred confiable
UDP	1980	RFC 768	Sin conexión	Puertos y checksum; 8 bytes
TCP	1981	RFC 793	Con conexión	Flujo de bytes confiable, 3 vías
TCP Tahoe	1988	(Jacobson)	Con conexión	Slow start, fast retransmit
TCP Reno	1990	RFC 2581	Con conexión	Agrega fast recovery

Protocolo	Año	RFC	Orientación	Característica principal
DCCP	2006	RFC 4340	Con conexión, no confiable	Congestión sin confiabilidad
SCTP	2007	RFC 4960	Con conexión	Multiflujo y multihoming
MPTCP	2020	RFC 8684	Con conexión	Una conexión sobre varios caminos
QUIC	2021	RFC 9000	Con conexión	Sobre UDP, TLS 1.3 integrado
TCP	2022	RFC 9293	Con conexión	Consolida y reemplaza al RFC 793

Se distinguen tres etapas. La fundacional (1970-1981) define los dos servicios básicos que todavía estructuran la capa. La segunda, iniciada tras el colapso por congestión de 1986, cuando el rendimiento de algunos enlaces cayó de 32 kbps a 40 bps, está dominada por el control de congestión: Van Jacobson dio origen a Tahoe y Reno, que evolucionaron hacia CUBIC y BBR. La tercera busca nuevos servicios: SCTP para señalización telefónica, DCCP para multimedia, MPTCP para dispositivos con varias interfaces y QUIC, que replantea el transporte sobre UDP. El RFC 9293 no redefine TCP, sino que consolida cuatro décadas de extensiones sobre un diseño básico que sigue vigente; y la escasa adopción de SCTP y DCCP, bloqueados con frecuencia por NAT y firewalls, explica la estrategia de QUIC.

3. Servicios y Primitivas de la Capa de Transporte

Esta sección analiza la capa desde el punto de vista de sus usuarios: qué servicios ofrece, cómo se organiza y mediante qué primitivas se accede a ella.

3.1. Servicio Orientado a Conexión y Servicio sin Conexión

Al igual que en la capa de red, existen dos tipos de servicio. El orientado a conexión se desarrolla en tres fases: establecimiento, en la que ambos extremos crean el estado necesario —números de secuencia, buffers, temporizadores—; transferencia, en la que el protocolo garantiza datos completos, sin duplicados y en orden, con control de errores y de flujo; y liberación. En Internet lo implementa TCP. Su costo es un retardo inicial de al menos un tiempo de ida y vuelta (Round-Trip Time, RTT) y el estado que el servidor mantiene por cliente.

El servicio sin conexión, implementado por UDP, es simple y sin controles: cada mensaje se envía de manera independiente, sin establecimiento ni garantía de entrega u orden. Su ventaja reside en lo que omite: no demora el primer dato ni introduce retardos por retransmisiones. Por eso se usa en el streaming en tiempo real, donde un fragmento de voz que llega tarde es tan inútil como uno perdido, y en consultas cortas como las de DNS.

Ambos servicios existen en la capa 4 aunque la capa de red ya ofrezca uno porque el servicio de red pertenece al operador y el de transporte al usuario. Si la red es sin conexión, como IP, el usuario no puede cambiar los routers, pero sí colocar encima un transporte confiable; a la inversa, una aplicación puede preferir un transporte sin conexión por razones de latencia.

3.2. La Entidad de Transporte, el Segmento y el Encapsulamiento

Al conjunto de software y hardware que constituye la capa 4 se lo denomina entidad de transporte. Suele residir en el núcleo del sistema operativo, pero puede encontrarse en bibliotecas de las aplicaciones, en la placa de red —las tarjetas con TCP Offload Engine— o en un proceso de usuario. QUIC ejemplifica el caso de biblioteca: forma parte del navegador, lo que permite actualizarlo al ritmo de la aplicación.

La unidad que intercambian dos entidades pares es el segmento, antes llamado Transport Protocol Data Unit (TPDU). El segmento queda anidado dentro del paquete de capa 3, que a su vez viaja dentro de la trama de capa 2. Para TCP sobre Ethernet, la estructura es: encabezado Ethernet (14 bytes), IPv4 (20 bytes), TCP (20 bytes), datos y verificación de trama (4 bytes). Como la unidad máxima de transmisión (MTU) de Ethernet es de 1.500 bytes, restando los encabezados IP y TCP queda el clásico tamaño máximo de segmento (MSS) de 1.460 bytes, que evita la fragmentación en la capa de red.

3.3. Primitivas del Servicio de Transporte

Las primitivas son las operaciones que la capa ofrece a sus usuarios, materializadas como llamadas al sistema o funciones de biblioteca, y se escriben de manera independiente de la red subyacente. Tanenbaum propone un conjunto mínimo de cinco primitivas para un servicio orientado a conexión:

Primitiva	Segmento enviado	Significado
LISTEN	(ninguno)	Bloquearse hasta que alguien intente conectarse
CONNECT	CONNECTION REQ.	Intentar establecer una conexión
SEND	DATA	Enviar información
RECEIVE	(ninguno)	Bloquearse hasta que llegue un segmento DATA
DISCONNECT	DISCONNECTIO N REQ.	Solicitar la liberación

En una aplicación con un servidor y varios clientes, el servidor ejecuta LISTEN y queda bloqueado. El cliente ejecuta CONNECT, que envía un CONNECTION REQUEST; la entidad del servidor lo desbloquea y responde CONNECTION ACCEPTED, con lo que la conexión queda establecida. Luego ambos intercambian datos con SEND y RECEIVE, y cualquiera ejecuta DISCONNECT al finalizar. LISTEN y RECEIVE no generan segmentos: solo modifican el estado local. La secuencia completa se representa con un diagrama de estados finitos, cuyas transiciones disparan las primitivas o la llegada de segmentos.

3.4. Sockets de Berkeley

En la capa 4 las primitivas se conocen como «sockets». Aparecieron en 1983 con Unix 4.2BSD de Berkeley y se convirtieron en la interfaz de red de prácticamente todos los sistemas operativos; en Windows existe una API equivalente, Winsock. Las primitivas para TCP son las siguientes:

Primitiva	Lado	Función
SOCKET	Ambos	Crea un extremo y reserva espacio en tablas
BIND	Servidor	Asigna dirección local (IP y puerto)
LISTEN	Servidor	Fija la cola de conexiones entrantes
ACCEPT	Servidor	Espera y acepta un pedido de conexión
CONNECT	Cliente	Intenta establecer una conexión
SEND / RECEIVE	Ambos	Envía o recibe datos
CLOSE	Ambos	Libera la conexión

El servidor ejecuta en orden SOCKET, BIND, LISTEN y ACCEPT. A diferencia del modelo abstracto, LISTEN no bloquea: reserva una cola, y los pedidos que la excedan se descartan. ACCEPT es la operación bloqueante: al llegar un pedido crea un socket nuevo con un descriptor bidireccional —a diferencia de los pipes de Unix— y lo devuelve, lo que permite delegar la conexión en un hilo y seguir esperando en el original. El cliente no necesita BIND, porque al servidor no le importa su puerto: el sistema le asigna uno efímero. Al terminar, ambos ejecutan CLOSE y la liberación es simétrica.

Para TCP, la API ofrece un flujo de bytes confiable (reliable byte stream). En modo sin conexión, CONNECT no envía segmentos, sino que fija la dirección remota, y SEND y RECEIVE intercambian datagramas entre pares. Protocolos posteriores, como SCTP (RFC 4960) o el Structured Stream Transport (SST) de Ford (2007), extienden levemente esta API.

3.5. Ejemplo de Interacción Cliente-Servidor

La bibliografía de la cátedra ilustra la programación de sockets con un servidor de archivos de Internet escrito en C. El servidor inicializa la estructura de direcciones, crea el socket, activa la opción SETSOCKOPT para poder reutilizar el puerto, lo asocia con BIND y ejecuta LISTEN. En su bucle principal se bloquea en ACCEPT; al aceptar una conexión lee el nombre del archivo, lo abre, lo copia bloque a bloque en el socket, cierra archivo y conexión, y vuelve a esperar.

El cliente se invoca con el nombre del servidor y la ruta del archivo. Convierte el nombre en dirección IP con GETHOSTBYNAME, que consulta el sistema de nombres de dominio (DNS), crea su socket y ejecuta CONNECT, lo que desencadena el acuerdo en tres vías de la sección 4.3. Establecida la conexión, escribe el nombre del archivo en el socket y lee bloques hasta que RECEIVE devuelve cero bytes, señal de que el servidor cerró; entonces cierra su propio extremo.

El ejemplo es deliberadamente pobre: atiende a un cliente por vez, de modo que uno lento bloquea a todos; no aplica seguridad; y asume que el nombre llega completo en un solo RECEIVE, lo que no está garantizado en un flujo de bytes, donde no se preservan las fronteras entre escrituras. Un servidor real atendería

cada conexión en un hilo o con entrada/salida no bloqueante, delimitaría los mensajes con un prefijo de longitud y cifraría la conexión con TLS.

4. Elementos de los Protocolos de Transporte

Los protocolos de transporte se asemejan a los de enlace: ambos resuelven control de errores, secuenciación y control de flujo. Pero en la capa de enlace dos nodos se comunican por un canal físico que no reordena ni almacena tramas, mientras que en transporte el canal es toda una red, con almacenamiento, retardos variables y caminos múltiples. Además, el destino debe direccionarse explícitamente y la cantidad de conexiones simultáneas es mucho mayor.

4.1. Direccionamiento: TSAP y Puertos

Para indicar con qué proceso remoto se desea comunicar, la capa define puntos finales denominados Transport Service Access Point (TSAP), que en Internet son los puertos; su equivalente en capa 3 es el Network Service Access Point (NSAP), la dirección IP. Un host con una sola IP usa simultáneamente muchos puertos. Son campos de 16 bits, con 65.536 valores por protocolo, en espacios independientes para TCP y UDP. La Internet Assigned Numbers Authority (IANA) los divide en tres rangos: bien conocidos (0 a 1.023), reservados a servicios estándar y que en Unix solo pueden usar procesos privilegiados; registrados (1.024 a 49.151), como el 3306 de MySQL; y dinámicos o efímeros (49.152 a 65.535), que el sistema asigna temporalmente a los clientes.

Puerto	Servicio	Transporte
20 / 21	FTP (datos / control)	TCP
22 / 23	SSH / Telnet	TCP
25	SMTP	TCP
53	DNS	UDP y TCP
67 / 68	DHCP	UDP
80 / 443	HTTP / HTTPS	TCP (443 también UDP)
111	Port mapper	TCP y UDP
123 / 161	NTP / SNMP	UDP

La elección del transporte acompaña la naturaleza del servicio: lo que exige integridad completa usa TCP; las consultas breves o sensibles al tiempo, UDP. DNS usa ambos —UDP para consultas, TCP para respuestas grandes—, y con HTTP/3 el puerto 443 también circula por UDP sobre QUIC.

Para descubrir el puerto de un servicio sin número fijo, la bibliografía describe dos procesos. El port mapper escucha en el puerto conocido 111 y asocia nombres de servicio con puertos: el cliente le pregunta por el servicio, recibe el número y luego se conecta a él. El server process evita mantener activos servidores que se usan raramente: un único proceso escucha en varios puertos a la vez, como un proxy, y al llegar una conexión invoca al servidor correspondiente y se la traspasa. En Unix se implementó con el demonio inetd, y una variante actual es la activación por sockets de systemd.

4.2. Multiplexación y Demultiplexación

En un host con una única dirección de red, todas las conexiones de capa 4 —Dropbox, Zoom, correo y WhatsApp a la vez— comparten el mismo enlace. La multiplexación consiste en recoger datos de varios sockets, agregarles el encabezado con sus puertos y entregarlos a la capa de red; la demultiplexación, en usar esos campos en el receptor para entregar cada segmento al socket correcto.

En UDP, un socket se identifica por la IP y el puerto de destino, y la aplicación lee el origen para responder. En TCP, cada conexión se identifica por una tupla de cinco elementos: protocolo, IP y puerto de origen, IP y puerto de destino. Así, un servidor web atiende miles de conexiones en el mismo puerto 443, distinguidas por la IP y el puerto efímero de cada cliente. Esto tiene consecuencias prácticas: un proxy que abre muchas conexiones hacia un mismo destino dispone de unos 16.000 puertos efímeros, y cada conexión cerrada retiene el suyo en el estado TIME_WAIT, por lo que con alta tasa de conexiones nuevas pueden agotarse; se mitiga reutilizando conexiones mediante pools o keep-alive.

La multiplexación inversa reparte una sola conexión de transporte sobre varios caminos de red para obtener más ancho de banda o tolerancia a fallas. La cátedra cita a SCTP, que usa varias direcciones IP de cada extremo (multihoming); MPTCP lleva la idea a TCP, permitiendo usar WiFi y red celular a la vez. También existe en

capa 2, como la Operación Multienlace de WiFi 7 analizada en el Informe N.º 2. En este sentido debe entenderse que «TCP no multiplexa»: sí demultiplexa por puertos, pero cada conexión es estrictamente unicast, sin multicast ni reparto entre varias rutas.

4.3. Establecimiento de la Conexión

Establecer una conexión parece sencillo: se envía un CONNECTION REQUEST y se espera un CONNECTION ACCEPTED. El problema es que la red puede almacenar y duplicar paquetes: un segmento retenido en una cola puede aparecer mucho después de que el emisor lo retransmitió. Evitar que estos duplicados retrasados se tomen por segmentos nuevos es el problema crítico. La cátedra lo ilustra con un cliente que ordena a su banco una transferencia: los paquetes demoran, el cliente reenvía el pedido y el banco, sin forma de reconocer la copia, transfiere dos veces.

Usar puertos desechables o recordar todos los identificadores de conexión usados no es viable. La solución práctica es limitar la vida de los paquetes —por diseño de la red, con un contador de saltos como el Time To Live de IPv4 o con marcas de tiempo— y definir un tiempo máximo de vida T , que incluye las confirmaciones; en Internet ronda los 120 segundos. Transcurrido T , todas las copias de un paquete han desaparecido. Entonces basta con no reutilizar un número de secuencia dentro de un período T . Tomlinson (1975) propuso tomar el número inicial de un reloj en tiempo real de cada host, que sigue avanzando aun tras un reinicio y no necesita estar sincronizado con los demás.

Para acordar los números iniciales, dado que el propio pedido podría ser un duplicado, se usa el acuerdo en tres vías (three-way handshake) de Tomlinson, refinado por Sunshine y Dalal (1978). El host 1 envía un pedido con su número x ; el host 2 confirma x y anuncia su número y ; el host 1 confirma y en su primer segmento.

```

Host 1                                     Host 2
|--- CR (seq = x) ----->|
|<-- ACK (seq = y, ack = x) -----|
|--- DATA (seq = x, ack = y) ----->|

Duplicado retrasado de un CR viejo:
|    CR (seq = x) viejo ----->|
|<-- ACK (seq = y, ack = x) -----|
|--- REJECT (ack = y) ----->|
(host 1 no pidió conexión: aborta)

```

Si llega un pedido duplicado de una conexión vieja, el host 1, que no intenta conectarse, rechaza la respuesta. Si además llega un segmento de datos viejo, este confirma un número distinto del y recién elegido y se descarta. Ninguna combinación de duplicados establece una conexión falsa, porque cada extremo exige que el otro confirme un número que acaba de elegir.

TCP usa este método con banderas SYN y ACK y números de 32 bits que cuentan bytes. El número inicial se elige de forma pseudoaleatoria (RFC 6528) para que un atacante no pueda predecirlo e inyectar segmentos. En redes rápidas surge otro problema: a 10 Gbps el espacio de 2^{32} bytes se recorre en unos 3,4 segundos, muy por debajo de T ; el mecanismo Protection Against Wrapped Sequence numbers (PAWS), del RFC 1323, usa la marca de tiempo de TCP como extensión del número de secuencia.

4.4. Liberación de la Conexión

Existen dos estilos de desconexión. La asimétrica funciona como el teléfono: si un extremo cuelga, la conexión termina para ambos, y los datos aún en tránsito se pierden. La simétrica trata la conexión como dos canales unidireccionales que se liberan por separado: un extremo puede dejar de enviar mientras sigue recibiendo. TCP adopta este modelo, cerrando cada sentido con un FIN y su ACK.

La desconexión simétrica enfrenta el problema de los dos ejércitos. Dos ejércitos azules rodean desde las colinas a un ejército blanco acampado en el valle; juntos vencen, por separado son derrotados. El azul 1 envía un mensajero por el valle, donde puede ser capturado, proponiendo atacar al amanecer. Aunque llegue, el azul 1 no sabe si fue recibido; si el azul 2 confirma, es él quien no sabe si su confirmación llegó. Siempre hay un último mensaje cuya llegada se ignora, y puede demostrarse que no existe protocolo que resuelva el problema: si el último mensaje

no fuera esencial podría eliminarse, y repitiendo el argumento se llegaría a un protocolo sin mensajes.

En la práctica se renuncia al acuerdo perfecto: cada lado decide independientemente cuándo desconectarse, mediante un acuerdo en tres vías con temporizadores. En el caso normal, el host 1 envía un DISCONNECTION REQUEST (DR) y arranca un temporizador, el host 2 responde con su DR y el host 1 confirma con un ACK; ambos liberan la conexión. Si se pierde el ACK final, el temporizador del host 2 expira y este libera igual. Si se pierde el DR del host 2, el host 1 reenvía su pedido. Si se pierden la respuesta y las N retransmisiones, el host 1 se rinde y el host 2 libera al expirar su temporizador. Además, cada extremo libera la conexión si no recibe segmentos durante un tiempo prudencial, por ejemplo si el enlace queda fuera de servicio. El costo es la posibilidad de conexiones semiabiertas, que TCP mitiga con el estado TIME_WAIT y con mensajes keep-alive.

4.5. Control de Errores y Control de Flujo

La capa de transporte emplea los mismos mecanismos que la de enlace, pero de extremo a extremo. Cada segmento lleva una suma de verificación (checksum), obligatoria en TCP —abarca también un pseudoencabezado con las direcciones IP— y opcional en UDP sobre IPv4. Aunque es más débil que el código de redundancia cíclica (CRC) de Ethernet, detecta lo que la capa de enlace no ve: un error producido dentro de un router, donde el CRC se recalcula sobre datos ya corrompidos.

La recuperación combina números de secuencia, confirmaciones y temporizadores: si no llega el acuse de recibo, el segmento se reenvía. Este esquema es Automatic Repeat reQuest (ARQ), en su variante de parada y espera o con ventana deslizante, que permite varios segmentos en tránsito. WiFi usa parada y espera en capa 2, porque su producto ancho de banda por retardo es bajo; Ethernet no retransmite y deja la reparación a la capa 4. Las conexiones de transporte, en cambio, requieren ventanas grandes: con 1 Gbps y 50 ms de RTT deben estar en vuelo 6,25 MB, mientras que la ventana original de TCP, de 65.535 bytes, limitaría el envío a unos 10,5 Mbps; de allí la opción de escalado de ventana.

Las ventanas grandes exigen gestionar buffers en ambos extremos. La cátedra presenta tres organizaciones: buffers de igual tamaño, simples pero ineficientes con segmentos de longitud variable; buffers de tamaño variable, más eficientes y complejos; y un buffer circular por conexión, tal vez la mejor solución para conexiones con mucha carga. Como la memoria disponible cambia, se usa una ventana dinámica: el receptor informa, con cada confirmación, el último segmento recibido y el espacio libre, a modo de créditos. Si se pierde el segmento que anuncia nuevo espacio, el emisor queda bloqueado; por eso cada host debe enviar periódicamente el estado de sus buffers, y TCP usa un temporizador de persistencia que sondea la ventana nula.

Cuando la memoria deja de ser el límite, aparece la capacidad de la red. La ventana dinámica debe ajustarse a ambas: TCP transmite según la menor entre la ventana anunciada por el receptor y la ventana de congestión, objeto de la sección 5.

4.6. Recuperación ante Caídas (Crash Recovery)

Si falla un router o un enlace, los extremos conservan su estado y retransmiten. El problema difícil es la caída del host receptor, que al reiniciarse pierde la información de sus conexiones. Supóngase un cliente que envía un archivo con parada y espera y un servidor que se cae y se reinicia, avisando a todos los hosts. Cada cliente está en el estado S1, con un segmento sin confirmar, o S0, sin pendientes, y puede retransmitir siempre, nunca, solo en S0 o solo en S1. El receptor, por su parte, confirma (A) y escribe (W) cada segmento en algún orden, y la caída (C) puede ocurrir en cualquier punto; los paréntesis indican operaciones no ejecutadas:

Emisor	AC(W)	AWC	C(AW)	C(WA)	WAC	WC(A)
Siempre	OK	Dup.	OK	OK	Dup.	Dup.
Nunca	Perd.	OK	Perd.	Perd.	OK	OK
En S0	OK	Dup.	Perd.	Perd.	Dup.	OK
En S1	Perd.	OK	OK	OK	OK	Dup.

Ninguna estrategia funciona en todos los casos: cada fila tiene al menos un segmento perdido o duplicado. Con la más intuitiva —retransmitir en S1— y un receptor que confirma antes de escribir, una caída entre ambas operaciones pierde

el segmento, porque el emisor ya recibió el ACK. La causa de fondo es que confirmar y escribir no pueden ejecutarse de forma atómica.

La conclusión, atribuida a Saltzer y colaboradores (1984), es que la recuperación ante la caída de un host nunca es transparente para las capas superiores: la recuperación de la capa N solo puede hacerse desde la N+1, si esta conserva suficiente estado. Por eso una aplicación como la bancaria de la sección 4.3 no puede delegar en TCP la ejecución exactamente única, y debe usar identificadores de operación, operaciones idempotentes o registros transaccionales.

5. Control de Congestión en la Capa de Transporte

El control de congestión es responsabilidad conjunta de las capas de red y de transporte: la congestión ocurre en los routers y la detecta la capa de red, pero la causa el tráfico de la capa de transporte, y solo se controla si las fuentes reducen su tasa. El objetivo es asignar a cada entidad de transporte una tasa de bits adecuada, logrando el mejor rendimiento sin congestionar la red.

5.1. Asignación Deseable del Ancho de Banda

La primera propiedad deseable es la eficiencia. El tráfico entregado (goodput) crece con la carga hasta acercarse a la capacidad; si la carga sigue aumentando, cae, porque las pérdidas y retransmisiones consumen capacidad sin aportar datos, y en el extremo se produce un colapso por congestión. El retardo, cuyo mínimo es el tiempo de propagación, crece abruptamente cerca de la capacidad. Kleinrock propuso en 1979 la métrica de potencia, tráfico / retardo, cuyo máximo indica el punto de operación ideal: la red bien aprovechada pero sin colas largas.

La segunda es la equidad. Como las redes no suelen reservar ancho de banda, es el control de congestión el que limita a cada fuente. El criterio más usado es la equidad máx-mín (max-min fairness), o «tasa justa»: una asignación lo es si no se puede aumentar la tasa de una fuente sin disminuir la de otra con tasa igual o menor. En el ejemplo de la cátedra, cuatro flujos comparten enlaces de capacidad 1; B, C y D atraviesan el enlace más cargado y reciben $1/3$ cada uno, mientras que A, que solo comparte otro enlace con B, recibe $1 - 1/3 = 2/3$. La equidad no implica tasas iguales, sino que nadie mejore a costa de alguien que esté peor. Como la

equidad se mide por conexión, aplicaciones como BitTorrent abren varias conexiones para obtener una porción mayor.

La tercera es la convergencia: las conexiones se abren y cierran dinámicamente, y el algoritmo debe alcanzar rápidamente la nueva asignación justa y eficiente después de cada cambio, sin oscilar en exceso.

5.2. Regulación de la Tasa de Envío: AIMD

Cada fuente necesita una señal del estado de la red. La cátedra lo ilustra con una analogía hidráulica: una red de gran capacidad que alimenta a un receptor con poco buffer plantea un problema de control de flujo; una red congestionada que alimenta a un receptor amplio, uno de asignación de tasa. Las señales posibles son la indicación explícita de la tasa por los routers, la notificación de congestión —como Explicit Congestion Notification (ECN, RFC 3168), que marca paquetes en lugar de descartarlos— o la inferencia a partir del retardo o la pérdida. Esta última es la señal clásica de TCP, porque en redes cableadas casi toda pérdida se debe a colas desbordadas.

Chiu y Jain demostraron en 1989 que, con una señal binaria, la ley que converge a un punto eficiente y justo es el incremento aditivo con decremento multiplicativo (Additive Increase Multiplicative Decrease, AIMD): sin congestión, cada fuente suma una constante a su tasa; con congestión, la multiplica por un factor menor que uno, típicamente $1/2$.

En su diagrama, la tasa del usuario 1 va en el eje horizontal y la del usuario 2 en el vertical. La recta de eficiencia une los puntos cuya suma es la capacidad; la de equidad es la diagonal a 45° desde el origen; el óptimo es su intersección. Un incremento aditivo mueve el punto a 45° , paralelo a la recta de equidad; un decremento multiplicativo lo mueve hacia el origen. Con AIMD, el punto sube hasta cruzar la recta de eficiencia y luego ambas tasas se reducen a la mitad, lo que achica la diferencia entre usuarios —quien más envía, más pierde—, mientras que el incremento aditivo la mantiene. Con capacidad 10 y tasas iniciales 8 y 2, la reducción lleva a 4 y 1 (diferencia 3); suben hasta 6,5 y 3,5 y bajan a 3,25 y 1,75 (diferencia 1,5): cada ciclo la diferencia se reduce a la mitad.

Las demás combinaciones fracasan. Con AIAD, ambos movimientos son paralelos a la recta de equidad y la diferencia nunca se reduce; con MIMD, ambos siguen rectas por el origen y la proporción entre tasas se conserva. Solo AIMD combina un movimiento que conserva la diferencia con otro que la reduce.

TCP aplica AIMD: la ventana de congestión crece un segmento por RTT y se reduce a la mitad ante una pérdida, en un perfil de «diente de sierra». No es del todo justo: como ajusta la ventana una vez por RTT, las conexiones más cortas obtienen más ancho de banda. Los demás protocolos deben ser «amigables con TCP» (TCP-friendly) para no apropiarse del enlace. Las variantes actuales son CUBIC TCP en Linux, basado en pérdidas con crecimiento cúbico; Compound TCP en Windows, que combina pérdida y retardo; y FAST TCP, que usa el retardo de propagación como métrica.

5.3. Congestión y Pérdidas en Redes Inalámbricas y Satelitales

Los protocolos de transporte deberían ser independientes de las capas inferiores, pero las redes inalámbricas contradicen el supuesto de que toda pérdida indica congestión: allí la mayoría se debe a interferencias y desvanecimiento. Una conexión TCP rápida tolera alrededor de un 1 % de pérdida, mientras que en WiFi es normal un 10 %. La fórmula de Mathis aproxima el rendimiento de AIMD como $1,22 \times \text{MSS} / (\text{RTT} \times \sqrt{p})$; con MSS de 1.460 bytes y RTT de 100 ms, una pérdida del 1 % permite unos 1,42 Mbps y una del 10 %, apenas 0,45 Mbps, sin importar la capacidad del enlace.

La solución es ocultar las pérdidas retransmitiendo en capa 2: WiFi usa parada y espera y reintenta varias veces antes de notificar una pérdida. Funciona por la escala de tiempos: la retransmisión de capa 2 ocurre en milisegundos y la de capa 4 en alrededor de un segundo, unas mil veces más. En una conexión con un tramo cableado y otro inalámbrico, las pérdidas del último tramo se reparan localmente y TCP solo percibe un leve aumento del retardo. Otras propuestas son el protocolo Snoop, que retransmite desde la estación base, y la división de la conexión con un intermediario (Performance Enhancing Proxy, RFC 3135), que rompe la semántica extremo a extremo.

En las redes satelitales el problema es el retardo: un satélite geoestacionario orbita a 35.786 km y el RTT supera los 500 ms. El tiempo de retransmisión de capa 2 es

comparable al de capa 4, por lo que ambas reaccionarían al mismo error; se usa entonces corrección de errores hacia adelante (Forward Error Correction, FEC) o se renuncia a retransmitir en transporte. Además, AIMD tarda mucho en alcanzar la ventana óptima, que debe ser enorme: con 50 Mbps y 600 ms deben estar en vuelo 3,75 MB. Las constelaciones de órbita baja, como Starlink, con RTT de 25 a 60 ms, atenúan el problema. Estos casos muestran los límites de la independencia entre capas, y buena parte de la evolución de los algoritmos de congestión puede leerse como el esfuerzo por distinguir la congestión real de las pérdidas propias de cada tecnología de acceso.

6. Protocolo UDP (User Datagram Protocol)

6.1. Características y Modelo de Servicio

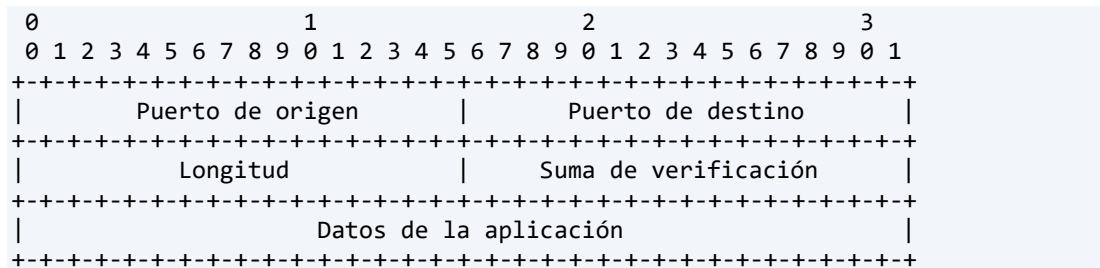
El Protocolo de Datagramas de Usuario (UDP, User Datagram Protocol) es el protocolo de transporte no orientado a conexión de la arquitectura TCP/IP, especificado por Jon Postel en la RFC 768 de 1980, un documento de apenas tres páginas. Su servicio consiste esencialmente en el de IP (Internet Protocol) más dos agregados: la multiplexación por puertos, analizada en la sección 4.2, y una suma de verificación que permite detectar datos corrompidos.

UDP no establece conexión, de modo que el primer datagrama ya transporta datos, y no mantiene estado en los extremos, por lo que un servidor puede atender a una gran cantidad de clientes sin reservar buffers por cada uno. No numera, confirma ni retransmite datagramas: no garantiza la entrega ni el orden. Tampoco aplica control de flujo ni de congestión, y si un datagrama llega con la suma de verificación incorrecta lo descarta sin aviso al emisor. A diferencia de TCP, preserva los límites de los mensajes: cada escritura de la aplicación genera exactamente un datagrama y cada lectura devuelve un datagrama completo.

Estas ausencias trasladan la responsabilidad a la aplicación, que implementa exactamente la confiabilidad que necesita; por eso QUIC (sección 9.3) se construye sobre UDP. Como contrapartida, una aplicación sin control de congestión propio puede perjudicar a los flujos TCP que compartan el enlace, y la RFC 8085 le pide un comportamiento «TCP-friendly», en el sentido de la sección 5.2.

6.2. Estructura del Encabezado UDP

El encabezado UDP tiene una longitud fija de 8 bytes, organizados en cuatro campos de 16 bits, como muestra el siguiente diagrama en el formato habitual de las RFC:



La función de cada campo se resume en la tabla siguiente:

Campo	Tamaño	Función
Puerto de origen	16 bits	Proceso emisor; opcional, vale 0 si no se usa
Puerto de destino	16 bits	Proceso receptor en el host destino
Longitud	16 bits	Encabezado más datos, en bytes; mínimo 8
Suma de verificación	16 bits	Cubre pseudoencabezado, encabezado y datos

El campo longitud limita el datagrama a 65.535 bytes (65.507 de datos sobre IPv4), pero en la práctica se evitan datagramas mayores que la MTU (Maximum Transmission Unit) del camino, porque perder un fragmento IP hace perder el datagrama entero; por eso DNS limitó históricamente sus respuestas a 512 bytes. Con 8 bytes frente a los 20 mínimos de TCP, UDP impone la menor sobrecarga posible a mensajes pequeños.

6.3. Pseudoencabezado y Suma de Verificación

La suma de verificación es el complemento a uno de la suma en complemento a uno de las palabras de 16 bits del encabezado (con el campo de suma en cero), de los datos (rellenados con un byte nulo si su longitud es impar) y de un pseudoencabezado que no se transmite, sino que ambos extremos construyen con información de la capa de red. Incluir las direcciones IP y el número de protocolo permite detectar datagramas entregados al host o al protocolo equivocado; es una violación deliberada de la independencia entre capas, que TCP replica con su propio número de protocolo. El pseudoencabezado difiere según la versión de IP:

Elemento	IPv4 (RFC 768)	IPv6 (RFC 8200)
Direcciones de origen y destino	32 bits cada una	128 bits cada una
Longitud	16 bits	32 bits
Relleno de ceros	8 bits	24 bits
Protocolo	8 bits, valor 17	8 bits Next Header, valor 17
Tamaño total	12 bytes	40 bytes

Más que el tamaño, importa la obligatoriedad. En IPv4 la suma es opcional, porque el encabezado IPv4 tiene la suya: el valor 0 indica que no se calculó, y si el cálculo da 0 se transmite 0xFFFF. IPv6 eliminó la suma del encabezado de red, de modo que la de UDP pasó a ser la única protección de las direcciones y es obligatoria, salvo en túneles (RFC 6935 y 6936). Es un mecanismo débil, que se apoya en el CRC (Cyclic Redundancy Check) de enlace y sirve sobre todo para detectar errores introducidos dentro de los routers.

6.4. Aplicaciones sobre UDP: DNS, DHCP, RPC y RTP/RTCP

El Sistema de Nombres de Dominio (DNS, Domain Name System) es el uso canónico de UDP: una consulta y una respuesta de pocos cientos de bytes al puerto 53 resuelven un nombre en un solo RTT (Round Trip Time, tiempo de ida y vuelta), mientras que TCP agregaría el establecimiento y la liberación de la conexión. Recurre a TCP solo cuando la respuesta no cabe en un datagrama y para las transferencias de zona.

El Protocolo de Configuración Dinámica de Host (DHCP, Dynamic Host Configuration Protocol) usa UDP porque el cliente todavía no tiene dirección IP. El intercambio DISCOVER, OFFER, REQUEST y ACK se realiza entre los puertos 68 (cliente) y 67 (servidor), y el primer mensaje se envía en difusión a 255.255.255.255 desde 0.0.0.0, algo imposible con TCP, que no admite difusión.

La Llamada a Procedimiento Remoto (RPC, Remote Procedure Call), de Birrell y Nelson (1984), hace que invocar una función remota parezca local: un stub empaqueta los parámetros y espera la respuesta. ONC RPC de Sun y NFS (Network File System) usaron UDP, con el port mapper en el puerto 111. Si la respuesta no llega, el cliente no sabe si el procedimiento se ejecutó: repetir es

inocuo en operaciones idempotentes, como leer un bloque, pero no en un débito bancario.

El Protocolo de Transporte en Tiempo Real (RTP, Real-time Transport Protocol, RFC 3550) soporta la voz sobre IP y la videoconferencia. Corre en la aplicación sobre UDP, pero su encabezado de 12 bytes cumple funciones de transporte: tipo de carga útil (el códec), número de secuencia, marca de tiempo e identificador de fuente (SSRC). No retransmite, porque una muestra tardía es inútil; el receptor absorbe la variación del retardo con un buffer de reproducción (jitter buffer). El Protocolo de Control de RTP (RTCP, RTP Control Protocol) transporta informes de pérdidas, jitter y retardo con los que el emisor adapta la tasa de codificación, un control de congestión implementado en la aplicación.

7. Protocolo TCP (Transmission Control Protocol)

7.1. Modelo de Servicio de TCP

El Protocolo de Control de Transmisión (TCP, Transmission Control Protocol) fue diseñado para brindar una conexión confiable de extremo a extremo sobre redes poco confiables. Su especificación original es la RFC 793 de 1981; las extensiones posteriores fueron tantas que la RFC 4614 se publicó como guía de todas las RFC que intervienen en TCP, y en 2022 la RFC 9293 consolidó la especificación base. Como en UDP, cada entidad se identifica por un puerto de 16 bits (sección 4.1), y los procesos servidores se ejecutan en segundo plano como daemons: el daemon de FTP, por ejemplo, usa los puertos 20 y 21.

El servicio es de flujo de bytes (byte-stream), no de mensajes: TCP agrupa los bytes en segmentos sin preservar los límites de las escrituras. Si un proceso escribe 2.048 bytes en cuatro operaciones de 512, pueden viajar como cuatro segmentos de 512, dos de 1.024 o uno de 2.048, y el receptor no puede saber cómo se partitionaron. Por eso los protocolos de aplicación sobre TCP delimitan sus propios mensajes, con prefijos de longitud o separadores.

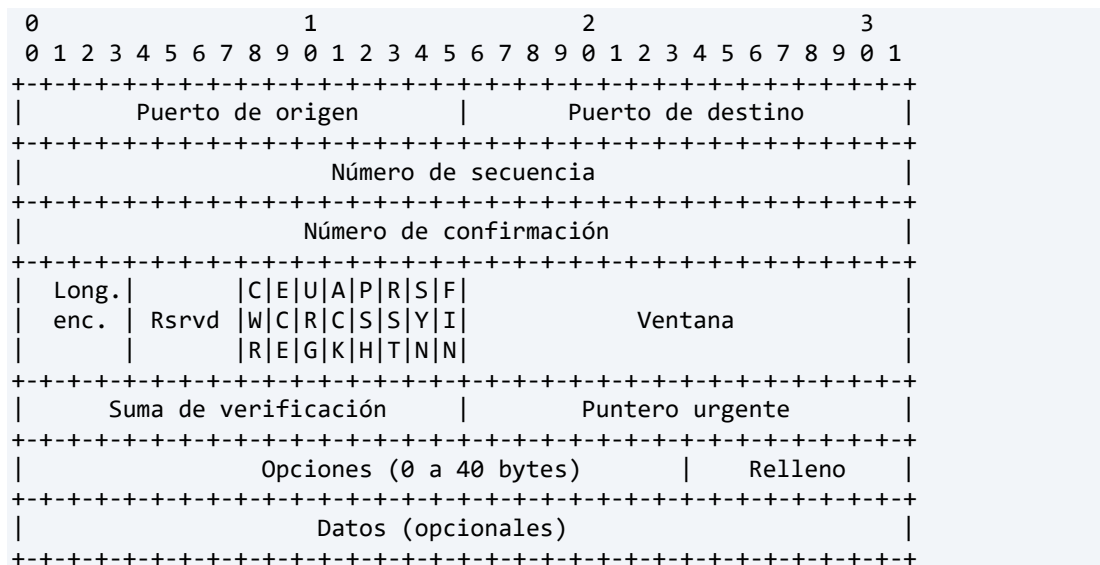
Las conexiones son full-duplex, unicast y punto a punto: los datos fluyen en ambos sentidos con números de secuencia y ventanas independientes, y cada conexión tiene exactamente dos extremos, sin soporte de multidifusión ni difusión. Cada

conexión se identifica por la cuádrupla de direcciones IP y puertos. Un segmento podría alcanzar los 64 KB de carga útil de IP, pero en la práctica no supera los 1.460 bytes de datos: la MTU de Ethernet de 1.500 bytes menos 20 de encabezado IPv4 y 20 de TCP, para evitar la fragmentación.

La bandera PSH pide al receptor entregar los datos sin esperar a llenar el buffer. Los datos urgentes, marcados con URG y el puntero urgente, señalan información a procesar fuera de orden, como la interrupción de un comando en Telnet; por discrepancias entre implementaciones, la RFC 6093 desaconseja usarlos. No deben confundirse con la opción TCP_NODELAY, que desactiva el algoritmo de Nagle (sección 7.7).

7.2. Estructura del Segmento TCP

Todo segmento comienza con un encabezado fijo de 20 bytes, ampliable con hasta 40 bytes de opciones, seguido de datos que pueden estar ausentes, como en las confirmaciones puras o en los segmentos de establecimiento y liberación:



La tabla siguiente detalla los campos:

Campo	Bits	Función
Puertos de origen y destino	16 + 16	Identifican los procesos extremos
Número de secuencia	32	Posición del primer byte de datos
Número de confirmación	32	Próximo byte esperado; válido si ACK = 1
Longitud del encabezado	4	Tamaño en palabras de 32 bits

Campo	Bits	Función
Reservado	4	Sin uso; vale cero
Banderas	8	CWR, ECE, URG, ACK, PSH, RST, SYN, FIN
Ventana	16	Bytes que el receptor acepta
Suma de verificación	16	Obligatoria; incluye el pseudoencabezado
Puntero urgente	16	Fin de los datos urgentes
Opciones	0 a 320	MSS, escala, marcas de tiempo, SACK

Los números de secuencia cuentan bytes, no segmentos: tras un segmento con secuencia 1.000 y 500 bytes, el siguiente comienza en 1.500, y una confirmación de 1.500 indica que llegaron todos los anteriores. SYN y FIN consumen un número de secuencia cada uno. La longitud del encabezado va de 5 a 15 palabras (20 a 60 bytes), y la suma se calcula como en UDP, con protocolo 6 en el pseudoencabezado.

7.3. Banderas de Control y Opciones

Las ocho banderas de un bit definen la función de cada segmento. A las seis de la RFC 793 se sumaron CWR y ECE, tomadas del campo reservado por la RFC 3168 para la notificación explícita de congestión:

Bandera	Significado cuando vale 1
CWR (Congestion Window Reduced)	El emisor redujo su ventana tras recibir ECE
ECE (ECN-Echo)	El receptor avisa que la red marcó congestión
URG (Urgent)	El puntero urgente es válido
ACK (Acknowledgement)	El número de confirmación es válido
PSH (Push)	Entregar los datos a la aplicación sin demora
RST (Reset)	Abortar la conexión por error o estado inválido
SYN (Synchronize)	Sincronizar números de secuencia al abrir
FIN (Finish)	El emisor no tiene más datos

Casi todos los segmentos llevan ACK en 1, porque confirman los datos del sentido opuesto (piggybacking); solo el SYN inicial del cliente lo lleva en 0. Las opciones,

con formato tipo-longitud-valor, añaden funciones ausentes del encabezado fijo; las más usadas son cuatro:

- **Tamaño máximo de segmento (MSS, Maximum Segment Size):** cada extremo anuncia en su SYN cuántos bytes de datos acepta por segmento; el valor puede diferir en cada sentido. Si no se envía, se asume por defecto 536 bytes de datos (RFC 879 y RFC 1122), correspondientes al datagrama IP de 576 bytes que todo host debe aceptar. El apunte de la cátedra expresa ese valor como 556 bytes porque suma los 20 del encabezado TCP. Sobre Ethernet el MSS habitual es 1.460 bytes en IPv4 y 1.440 en IPv6.
- **Escala de ventana (Window Scale, RFC 7323):** la ventana de 16 bits limita los datos sin confirmar a 64 KB. En una línea de 600 Mbps esa ventana se transmite en menos de 1 ms y, con 50 ms de propagación transoceánica, la línea queda ociosa más del 98 % del tiempo. La opción, negociada en los SYN, desplaza el valor de la ventana de 0 a 14 bits y permite ventanas de hasta 2^{30} bytes, es decir, 1 GB.
- **Marcas de tiempo (Timestamps, RFC 7323):** cada segmento lleva su instante de envío y el eco de la última marca recibida, lo que permite medir el RTT incluso sobre retransmisiones y habilita PAWS (Protection Against Wrapped Sequence numbers): a 10 Gbps el espacio de secuencia de 32 bits se recorre en unos 3,4 s, y la marca permite descartar segmentos viejos.
- **SACK permitido (SACK-permitted, RFC 2018):** indica en el SYN que se admiten confirmaciones selectivas (sección 7.9).

7.4. Establecimiento de la Conexión: Three-Way Handshake

TCP usa el acuerdo de tres vías (three-way handshake) de Tomlinson, cuyos fundamentos frente a los duplicados retrasados se vieron en la sección 4.3. El servidor ejecuta LISTEN y ACCEPT y queda a la espera. La primitiva CONNECT del cliente envía un segmento con SYN = 1 y ACK = 0, su número de secuencia inicial (ISN, Initial Sequence Number) x y sus opciones, entre ellas el MSS. El servidor responde con SYN = 1, ACK = 1, su propio ISN y y confirmación $x + 1$. El cliente cierra el acuerdo con ACK = 1, secuencia $x + 1$ y confirmación $y + 1$, segmento que ya puede llevar datos. Cada extremo respeta el MSS anunciado por

el otro, y el descubrimiento de la MTU del camino (Path MTU Discovery) completa la información sobre los enlaces intermedios.

La RFC 793 derivaba el ISN de un reloj incrementado cada 4 μ s, esquema predecible que Kevin Mitnick explotó en 1994 para falsificar conexiones; desde la RFC 6528 se le suma un hash de la cuádrupla y de una clave secreta, lo que lo vuelve impredecible. En la apertura simultánea, dos hosts se envían un SYN a la vez, pasan por SYN_SENT y SYN_RECEIVED y llegan a ESTABLISHED tras cuatro segmentos, formando una única conexión, como señala el apunte.

Al recibir un SYN, el servidor reserva recursos antes de saber si el cliente es legítimo. La inundación SYN (SYN flood) envía miles de SYN con direcciones falsificadas hasta agotar la cola de conexiones pendientes. Entre las contramedidas de la RFC 4987, citada en el apunte, se destacan las SYN cookies de Daniel Bernstein (1996): el servidor no guarda estado y codifica en su ISN un contador de tiempo, un índice de MSS y un hash criptográfico de la cuádrupla con una clave secreta; al llegar el ACK final recalcula el hash y, si coincide, recién entonces crea la conexión. Es el uso de criptografía en el establecimiento que menciona el apunte.

7.5. Liberación de la Conexión

La liberación es simétrica: cada extremo cierra por separado su sentido de transmisión enviando un segmento con FIN = 1, que el otro confirma con ACK. El sentido opuesto puede seguir transportando datos hasta que el otro extremo envíe su propio FIN, de modo que la liberación completa insume cuatro segmentos, o tres si el ACK del primer FIN y el segundo FIN viajan juntos. El estado intermedio se denomina semicierre (half-close): un cliente puede enviar todos sus datos, ejecutar shutdown con SHUT_WR y seguir recibiendo los resultados por el sentido aún abierto.

Frente al problema de los dos ejércitos, discutido en la sección 4.4, TCP adopta una solución pragmática basada en temporizadores. El extremo que cierra primero, tras confirmar el FIN ajeno, permanece en TIME_WAIT durante dos veces el tiempo máximo de vida de un segmento (MSL, Maximum Segment Lifetime). La RFC 793 fijó el MSL en 2 minutos. TIME_WAIT permite retransmitir el último ACK si se perdió y garantiza que los segmentos retrasados desaparezcan antes de que una nueva conexión reutilice la misma cuádrupla.

La liberación abrupta usa la bandera RST, que aborta la conexión de inmediato, sin TIME_WAIT y descartando los datos pendientes. Se genera ante un segmento dirigido a un puerto sin proceso escuchando (la respuesta típica a un escaneo de puertos cerrados), ante segmentos de una conexión desconocida, por ejemplo tras un reinicio, o por decisión de la aplicación.

7.6. Máquina de Estados Finitos de TCP

El establecimiento y la liberación se especifican como una máquina de once estados, cuyas transiciones obedecen a llamadas de la aplicación, a la llegada de segmentos o a la expiración de temporizadores:

Estado	Significado
CLOSED	No hay conexión activa ni pendiente
LISTEN	El servidor espera un SYN
SYN_SENT	Se envió SYN; se espera SYN+ACK
SYN_RECEIVED	Llegó SYN, se envió SYN+ACK; se espera ACK
ESTABLISHED	Transferencia normal de datos
FIN_WAIT_1	Se envió FIN; se espera su ACK
FIN_WAIT_2	FIN propio confirmado; se espera el FIN ajeno
CLOSE_WAIT	Llegó el FIN ajeno; se espera el cierre local
LAST_ACK	Se envió el FIN tras CLOSE_WAIT; se espera ACK
CLOSING	Ambos extremos enviaron FIN simultáneamente
TIME_WAIT	Espera de $2 \cdot \text{MSL}$ antes de liberar

El recorrido típico del cliente, trazado con líneas gruesas en el diagrama de Tanenbaum, es CLOSED, SYN_SENT y ESTABLISHED al abrir; al cerrar, FIN_WAIT_1, FIN_WAIT_2 y TIME_WAIT, desde donde vuelve a CLOSED al vencer el temporizador. El servidor sigue el camino complementario: LISTEN, SYN_RECEIVED y ESTABLISHED; al recibir el FIN del cliente pasa a CLOSE_WAIT, donde puede permanecer indefinidamente si la aplicación nunca cierra el socket, y luego a LAST_ACK y CLOSED.

7.7. Ventana Deslizante y Control de Flujo en TCP

El control de flujo aplica sobre los mecanismos de buffers y ARQ (Automatic Repeat reQuest) de la sección 4.5 una ventana deslizante de tamaño variable en bytes. En cada segmento el receptor anuncia cuántos bytes acepta a partir del número de confirmación, según el espacio libre en su buffer, y el emisor no puede tener más que eso sin confirmar. En el ejemplo del apunte, un receptor con buffer de 4 KB recibe segmentos de 2 KB: tras el primero anuncia una ventana de 2 KB y tras el segundo una ventana cero, que detiene al emisor hasta que la aplicación lea datos y el receptor envíe una actualización de ventana.

Si esa actualización se pierde, ambos extremos quedan esperándose indefinidamente, porque los segmentos sin datos no se retransmiten. Para evitarlo, el emisor activa ante la ventana cero el temporizador de persistencia y, al vencer, envía una sonda de ventana (window probe) de un byte que fuerza al receptor a responder con su ventana actual.

El otro problema son los segmentos diminutos: en el ejemplo de Tanenbaum de un editor remoto sobre Telnet, cada tecla genera 162 bytes en cuatro segmentos. En el receptor, la confirmación retardada (delayed ACK) espera hasta 500 ms (RFC 1122) para combinar la confirmación con la actualización de ventana o con datos de respuesta. En el emisor actúa el algoritmo de Nagle (RFC 896, 1984), que el apunte presenta como el modo de enviar segmentos grandes cuando la aplicación tolera retardo: mientras haya datos sin confirmar, los datos pequeños nuevos se acumulan hasta la confirmación o hasta completar un MSS. En juegos o escritorios remotos resulta perjudicial y se desactiva con TCP_NODELAY.

El problema simétrico es el síndrome de la ventana tonta (silly window syndrome), descrito por David Clark en la RFC 813 de 1982: si la aplicación receptora lee de a un byte, el receptor anuncia ventanas de un byte y el emisor envía segmentos de un byte de datos. La solución de Clark impide anunciar ventanas pequeñas: tras una ventana cero, la actualización solo se envía cuando cabe un MSS o se liberó la mitad del buffer.

7.8. Administración de Temporizadores

El temporizador más importante es el de retransmisión, cuyo valor (RTO, Retransmission Timeout) es más difícil de fijar que en la capa de enlace: allí el tiempo de confirmación tiene una distribución estrecha, de microsegundos,

mientras que en transporte el RTT atraviesa colas variables y su distribución es amplia y cambiante. Como señala el apunte, un RTO grande demora la reacción ante pérdidas y uno chico genera retransmisiones innecesarias, por lo que debe ajustarse dinámicamente.

El algoritmo de Van Jacobson (1988), especificado en la RFC 6298, que reemplazó a la RFC 2988 citada en el apunte, mantiene el RTT suavizado (SRTT, Smoothed RTT) y su variación (RTTVAR, RTT Variation), estimada con la desviación media por ser más simple que la estándar. La primera medición R inicializa $SRTT = R$ y $RTTVAR = R/2$; con cada medición R' posterior se calcula:

$$\begin{aligned} RTTVAR &= (1 - \beta) \cdot RTTVAR + \beta \cdot |SRTT - R'| && \text{con } \beta = 1/4 \\ SRTT &= (1 - \alpha) \cdot SRTT + \alpha \cdot R' && \text{con } \alpha = 1/8 \\ RTO &= SRTT + \max(G, 4 \cdot RTTVAR) \end{aligned}$$

G es la granularidad del reloj. Frente al $RTO = 2 \cdot SRTT$ de la RFC 793, el término $4 \cdot RTTVAR$ hace crecer el RTO cuando el retardo se vuelve errático. El RTO inicial y el mínimo recomendado son de 1 s, el valor que menciona el apunte, y cada expiración duplica el RTO (retroceso exponencial).

Cuando se confirma un segmento retransmitido no puede saberse a qué transmisión corresponde la confirmación. El algoritmo de Karn (1987) resuelve la ambigüedad excluyendo esas mediciones del SRTT y conservando el RTO duplicado hasta confirmar un segmento no retransmitido; la opción de marcas de tiempo elimina el problema de raíz. Además, TCP usa otros tres temporizadores:

- **Persistencia:** ante una ventana cero dispara las sondas de ventana descriptas en la sección 7.7.
- **Mantenimiento (keepalive):** sondea conexiones inactivas para liberar las de extremos caídos sin aviso; la RFC 1122 lo define opcional, desactivado por defecto y con un intervalo no menor a 2 horas.
- **TIME_WAIT:** mantiene la conexión durante $2 \cdot MSL$ tras el cierre activo (sección 7.5).

7.9. Control de Congestión en TCP

TCP controla la congestión desde los extremos aplicando los principios de asignación de la sección 5.1 y la ley AIMD de Chiu y Jain de la sección 5.2. El emisor mantiene una ventana de congestión ($cwnd$, congestion window) que estima

la capacidad disponible en la red, y puede tener sin confirmar el mínimo entre $cwnd$ y la ventana anunciada por el receptor. La señal clásica de congestión es la pérdida, supuesto que falla en enlaces inalámbricos (sección 5.3). El reloj de confirmaciones (ACK clock) del apunte ajusta el ritmo: si tras un tramo de 1 Gbps hay uno de 1 Mbps, las confirmaciones regresan con el espaciado del enlace lento y, como cada una habilita un envío, el emisor sigue al cuello de botella.

El arranque lento (slow start, Jacobson, 1988) parte de una ventana inicial de 1 MSS, hoy hasta 10 MSS (RFC 6928), y suma 1 MSS por confirmación, lo que la duplica en cada RTT. Al alcanzar el umbral $ssthresh$ (slow start threshold) se pasa a prevención de congestión (congestion avoidance), con un crecimiento de 1 MSS por RTT: la parte aditiva de AIMD.

Ante la expiración del RTO, $ssthresh$ pasa a la mitad de los datos en vuelo y $cwnd$ vuelve a 1 MSS. La retransmisión rápida (fast retransmit) evita esperar al RTO: tres confirmaciones duplicadas, que el receptor genera al recibir segmentos posteriores a uno perdido, se interpretan como pérdida y el segmento se retransmite de inmediato (se exigen tres para no confundir un reordenamiento). La recuperación rápida (fast recovery) asume que, si llegan duplicados, la congestión no es severa: $ssthresh$ pasa a la mitad, $cwnd$ a $ssthresh + 3 \text{ MSS}$, y al confirmarse la retransmisión $cwnd$ vuelve a $ssthresh$, produciendo el diente de sierra de AIMD. Las versiones de TCP, nombradas por las distribuciones de BSD, combinan estos mecanismos así:

Aspecto	Tahoe (1988)	Reno (1990)	NewReno (1999)
Ante 3 ACK duplicados	Retransmite; $cwnd = 1 \text{ MSS}$	Recuperación rápida desde $ssthresh$	Igual que Reno
Ante expiración del RTO	$cwnd = 1 \text{ MSS}$	$cwnd = 1 \text{ MSS}$	$cwnd = 1 \text{ MSS}$
Varias pérdidas por ventana	Reinicia el arranque lento	Suele terminar en RTO	Las resuelve con ACK parciales
Especificación vigente	Jacobson, 1988	RFC 5681	RFC 6582

Tahoe fija $ssthresh$ en la mitad pero vuelve a 1 MSS, creciendo exponencialmente hasta esa mitad, que es la que menciona el apunte. Reno continúa directamente

desde la mitad, pero sale de la recuperación con la primera confirmación nueva aunque cubra solo parte de lo pendiente; NewReno permanece en recuperación hasta cubrir todo lo que estaba en vuelo.

La confirmación selectiva (SACK, Selective Acknowledgement) resuelve el problema de raíz, porque la confirmación acumulativa solo informa el primer byte faltante. Definida en la RFC 2018 y extendida por la RFC 2883 citada en el apunte (D-SACK), informa hasta tres o cuatro rangos recibidos fuera de orden. En el ejemplo del apunte se envían los segmentos 1 a 6 y se pierden el 2 y el 5: el receptor confirma el 1 y declara por SACK el rango 3 a 4 y el 6, de modo que el emisor retransmite solo el 2 y el 5. La RFC 3517, actualizada por la RFC 6675, especifica la recuperación basada en SACK.

La notificación explícita de congestión (ECN, Explicit Congestion Notification, RFC 3168) detecta la congestión antes de que haya pérdidas. Negociada en el acuerdo de tres vías, usa dos bits del encabezado IP: un router con la cola creciendo, en lugar de descartar, marca el paquete con el código CE (Congestion Experienced). El receptor activa ECE en sus confirmaciones; el emisor reduce cwnd como ante una pérdida y responde con CWR, evitando la pérdida y la retransmisión.

7.10. Variantes Modernas: CUBIC, Compound TCP y BBR

Reno escala mal con un gran producto ancho de banda por retardo (BDP, Bandwidth-Delay Product): a 10 Gbps y 100 ms de RTT, recuperar la ventana tras reducirla a la mitad llevaría más de una hora. Las variantes modernas conservan la estructura de Reno pero cambian la forma de crecimiento.

TCP CUBIC (Ha, Rhee y Xu, 2008) es el algoritmo por defecto de Linux desde el núcleo 2.6.19, especificado en la RFC 8312 y actualizado por la RFC 9438. Su ventana depende del tiempo transcurrido desde la última pérdida y no de la cantidad de RTT:

$$W(t) = C \cdot (t - K)^3 + W_{\text{máx}}$$

$$K = \sqrt[3]{(W_{\text{máx}} \cdot (1 - \beta) / C)} \quad \text{con } C = 0,4 \text{ y } \beta = 0,7$$

$W_{\text{máx}}$ es la ventana al producirse la pérdida: la ventana se reduce al 70 %, crece rápido, se aplanan cerca de $W_{\text{máx}}$ y se acelera de nuevo si lo supera sin pérdidas. Al depender del tiempo, reduce la desventaja de las conexiones de RTT largo, aunque sigue basándose en pérdidas, como indica el apunte.

Compound TCP, de Microsoft (Windows Vista), combina pérdidas y retardo: suma una ventana de pérdidas, con reglas de Reno, y una de retardo, que crece mientras el RTT se mantiene cerca del mínimo. Las versiones recientes de Windows adoptaron CUBIC por defecto.

BBR (Bottleneck Bandwidth and Round-trip propagation time), presentado por Google en 2016, abandona la pérdida como señal: estima el ancho de banda del cuello de botella y el RTT mínimo, calcula el BDP y envía con espaciado temporal (pacing) a esa tasa, buscando el enlace lleno con las colas vacías en lugar de llenar los buffers (bufferbloat). Se emparenta con FAST TCP, que el apunte menciona por usar el tiempo de propagación como métrica.

8. Comparativa TCP vs. UDP

La elección entre TCP y UDP es una de las primeras decisiones de diseño de un protocolo de aplicación, y cada uno ofrece garantías distintas a cambio de costos en latencia, sobrecarga y complejidad. La tabla siguiente reúne las diferencias analizadas:

Característica	TCP	UDP
Especificación	RFC 793, hoy RFC 9293	RFC 768
Orientación	A conexión, tres vías	Sin conexión
Unidad de servicio	Flujo de bytes	Mensajes con límites
Confiabilidad y orden	Garantizados por retransmisión	No garantizados
Control de flujo	Ventana anunciada	No
Control de congestión	Slow start, AIMD, CUBIC, BBR	A cargo de la aplicación
Encabezado	20 a 60 bytes	8 bytes
Suma de verificación	Obligatoria	Opcional en IPv4, obligatoria en IPv6
Estado en los extremos	Buffers y temporizadores	Ninguno
Latencia inicial	1 RTT	Ninguna
Destinatarios	Unicast	Unicast, multicast y broadcast

TCP paga su canal confiable sobre todo con el bloqueo de cabeza de línea (head-of-line blocking): si se pierde un segmento, los datos posteriores ya recibidos quedan retenidos hasta la retransmisión. UDP elimina ese costo, pero no ofrece garantías.

El criterio es si la aplicación necesita integridad completa o frescura del dato. La web, el correo (SMTP, IMAP), la transferencia de archivos y el acceso remoto (SSH) requieren cada byte en orden, y el RTT de establecimiento se amortiza en conexiones persistentes: TCP es la elección natural. En la voz sobre IP el retardo de boca a oído no debería superar unos 150 ms, y una muestra retransmitida llegaría tarde: conviene descartarla, con RTP sobre UDP. Los juegos en línea siguen la misma lógica, porque cada actualización hace obsoleta a la anterior.

El streaming bajo demanda es intermedio: YouTube o Netflix usan HTTP sobre TCP, porque el buffer del reproductor tolera las retransmisiones, mientras que la televisión IP por multicast usa UDP. DNS ejemplifica las transacciones breves, donde establecer una conexión sería desproporcionado, y en el Internet de las cosas (IoT, Internet of Things) los sensores a batería aprovechan la ausencia de estado: CoAP (Constrained Application Protocol, RFC 7252) opera sobre UDP con confirmaciones propias. QUIC, por último, muestra que UDP puede ser el sustrato de un transporte confiable.

9. Protocolos de Transporte Modernos

9.1. SCTP: Multihoming y Multistreaming

El Protocolo de Transmisión de Control de Flujos (SCTP, Stream Control Transmission Protocol) fue desarrollado por el grupo SIGTRAN del IETF para transportar señalización telefónica SS7 sobre IP. Publicado en la RFC 2960 (2000), revisado en la RFC 4960 citada en el apunte y actualizado por la RFC 9260 (2022), opera directamente sobre IP con número de protocolo 132 y combina la confiabilidad y el control de congestión de TCP con la orientación a mensajes de UDP. Su asociación se establece con un acuerdo de cuatro vías (INIT, INIT-ACK, COOKIE-ECHO, COOKIE-ACK) en el que el servidor no guarda estado sino que devuelve una cookie firmada, lo que lo hace inmune por diseño a la inundación SYN. Los datos viajan en chunks protegidos con CRC-32c.

El multihoming permite que cada extremo tenga varias direcciones IP: una ruta es primaria y las demás se supervisan con mensajes HEARTBEAT, de modo que si la primaria falla el tráfico se conmuta sin cortar la asociación. El apunte presenta a SCTP, por esta capacidad, como ejemplo de la multiplexación inversa de la sección 4.2. El multistreaming permite múltiples flujos independientes dentro de una asociación, cada uno con su propio orden, de modo que una pérdida solo retiene los mensajes de su flujo; admite además entrega no ordenada y, con PR-SCTP, confiabilidad parcial. Como NAT y firewalls solo reconocen TCP y UDP, su uso se limita a la señalización de redes móviles 4G y 5G y a los canales de datos de WebRTC, encapsulado sobre UDP.

9.2. DCCP

El Protocolo de Control de Congestión de Datagramas (DCCP, Datagram Congestion Control Protocol, RFC 4340, 2006) cubre el hueco entre TCP y UDP: transporte no confiable, pero con control de congestión, ante la evidencia de que las aplicaciones multimedia sobre UDP rara vez lo implementaban. Es orientado a conexión y confirma los paquetes, pero no retransmite: las confirmaciones solo sirven para medir pérdidas. Permite elegir el algoritmo mediante un identificador (CCID, Congestion Control ID): el CCID 2 se comporta como TCP con AIMD, y el CCID 3 aplica TFRC (TCP-Friendly Rate Control, RFC 5348), que ajusta la tasa suavemente con la ecuación de rendimiento de TCP, más adecuado para audio y video. Como SCTP, chocó con la osificación de la red y su uso fue marginal, aunque su idea central, que todo transporte debe controlar la congestión, pasó a QUIC.

9.3. QUIC y HTTP/3

QUIC es el protocolo de transporte de propósito general más importante surgido desde TCP. Desarrollado por Google desde 2012, fue estandarizado por el IETF en 2021 en la RFC 9000, junto con la RFC 9001 (integración de TLS, Transport Layer Security) y la RFC 9002 (pérdidas y congestión); HTTP/3 se publicó en la RFC 9114 de 2022. Funciona sobre UDP, normalmente en el puerto 443, para eludir la osificación que frenó a SCTP y DCCP: un protocolo nuevo sobre IP sería bloqueado por NAT y firewalls, y modificar TCP exigiría actualizar el núcleo de miles de millones de sistemas. Sobre UDP, QUIC se implementa en el espacio de usuario y se actualiza al ritmo de la aplicación.

Una conexión HTTPS clásica necesita 1 RTT para TCP y otro para TLS 1.3. QUIC integra TLS 1.3 en su acuerdo: el primer paquete del cliente lleva los parámetros de transporte y el ClientHello, y la conexión cifrada queda lista en 1 RTT. Con una clave de reanudación de una conexión previa, el cliente puede enviar datos en el primer paquete (0-RTT), aunque esos datos pueden ser capturados y reenviados por un atacante, por lo que solo deben usarse en solicitudes idempotentes.

HTTP/2 multiplexa solicitudes sobre una conexión TCP, pero la pérdida de un segmento detiene a todas. QUIC, como SCTP, transporta flujos independientes y una pérdida solo retiene los flujos afectados; además, cada paquete lleva un número único y creciente incluso en las retransmisiones, lo que elimina la ambigüedad que en TCP resuelve el algoritmo de Karn. La migración de conexión es otra ventaja: QUIC identifica la conexión con identificadores (connection IDs) independientes de la cuádrupla, de modo que sobrevive al paso de un teléfono de WiFi a datos móviles. Por último, cifra los datos y casi todo el encabezado, incluidos los números de paquete, lo que protege la privacidad e impide que los dispositivos intermedios dependan de sus campos, a costa de visibilidad para los operadores y de más consumo de CPU. HTTP/3 está soportado por los navegadores principales y lo usa alrededor de un tercio de los sitios web.

9.4. MPTCP (Multipath TCP)

El TCP Multicamino (MPTCP, Multipath TCP), definido experimentalmente en la RFC 6824 (2013) y en su versión 1 en la RFC 8684 (2020), permite que una conexión use varios caminos a la vez, por ejemplo las interfaces WiFi y celular de un teléfono, para sumar capacidad o mantener respaldo. A diferencia de SCTP, se presenta a la aplicación como un socket TCP y a la red como conexiones TCP corrientes, llamadas subflujos. El SYN inicial lleva la opción MP_CAPABLE con claves; si el otro extremo no la soporta, la conexión sigue como TCP clásico. Los caminos adicionales se agregan con MP_JOIN, y un número de secuencia a nivel de datos, transportado en la opción DSS (Data Sequence Signal), permite reensamblar el flujo original.

Su control de congestión es acoplado: con algoritmos independientes por subflujo, dos caminos que comparten un cuello de botella obtendrían el doble que una conexión TCP. LIA (Linked Increases Algorithm, RFC 6356) coordina las ventanas

para que el conjunto no sea más agresivo que un flujo TCP en su mejor camino. Apple lo usa desde 2013 en Siri, y Linux lo incluye desde 2020.

9.5. Comparativa de Protocolos de Transporte

La tabla siguiente sintetiza los servicios que ofrece cada protocolo analizado:

Característica	TCP	UDP	SCTP	QUIC	MPTCP
Conexión	3 vías	No	4 vías con cookie	1-RTT o 0-RTT	3 vías por subflujo
Confiabilidad	Total	Ninguna	Total o parcial	Total por flujo	Total
Unidad de servicio	Bytes	Mensajes	Mensajes	Bytes por flujo	Bytes
Múltiples flujos	No	No	Sí	Sí	No
Bloqueo de cabeza de línea	Sí	No	Por flujo	Por flujo	Sí
Varias rutas	No	No	Respaldo	Migración	Simultáneas
Control de congestión	Sí	No	Sí	Sí	Acoplado
Cifrado integrado	No	No	No	TLS 1.3	No
Se apoya en	IP	IP	IP	UDP	TCP
Adopción	Universal	Universal	Telecomunicaciones	Web masiva	Móviles

Cada protocolo resuelve una limitación concreta de TCP: SCTP chocó con la osificación, MPTCP la eludió pareciendo TCP y QUIC se apoyó en UDP y cifró su encabezado. Los conceptos de TCP —ventana deslizante, confirmación selectiva, RTO adaptativo y AIMD— reaparecen en todos ellos.

10. Ejemplos Prácticos de Cálculo

Los mecanismos de la capa de transporte tienen consecuencias cuantitativas directas sobre el rendimiento que percibe una aplicación. El tamaño de la ventana, el tiempo de ida y vuelta, el temporizador de retransmisión o la proporción de bytes consumida en encabezados determinan cuánto tarda una descarga o cuánto ancho de banda ocupa una llamada. A continuación se presentan seis ejercicios resueltos paso a paso, del tipo que aparece en exámenes de la materia.

10.1. Throughput Máximo Limitado por la Ventana de Recepción

Enunciado

Una conexión TCP (Transmission Control Protocol) sin escalado de ventana tiene la ventana del receptor limitada al máximo del campo Window Size: 16 bits, es decir, 65.535 bytes («64 KB»). El tiempo de ida y vuelta (RTT, Round-Trip Time) es de 50 ms. Calcular el throughput máximo, compararlo con un enlace de 1 Gbps y determinar la ventana necesaria para aprovecharlo.

Resolución

El emisor no puede tener en vuelo más bytes que la ventana anunciada; tras enviarla debe esperar el primer ACK, que llega un RTT después. Si la ventana se transmite en menos de un RTT, el throughput queda acotado por Ventana / RTT:

- Ventana = 65.535 bytes $\times$ 8 = 524.280 bit.
- Throughput = 524.280 bit / 0,05 s = 10.485.600 bit/s $\approx$ 10,5 Mbps.
- Tiempo de transmisión de la ventana a 1 Gbps = 524.280 / $10^9 \approx$ 0,524 ms; el enlace queda ocioso los 49,5 ms restantes.
- Utilización = 10,49 / 1.000 $\approx$ 1,05 %.
- Ventana necesaria = 10^9 bit/s $\times$ 0,05 s = $5 \cdot 10^7$ bit = 6,25 MB, unas 95 veces el máximo del campo.

La siguiente tabla muestra el techo de throughput de una ventana de 65.535 bytes según el RTT:

RTT	Escenario típico	Throughput máximo
10 ms	Misma región	52,4 Mbps
50 ms	Distancia continental	10,5 Mbps
150 ms	Argentina hacia el exterior	3,5 Mbps
200 ms	Ruta transoceánica larga	2,6 Mbps

Sin escalado de ventana, el rendimiento de una conexión TCP depende de la distancia y no de la velocidad del enlace: un acceso de 1 Gbps rinde lo mismo que uno de 20 Mbps frente a un servidor a 50 ms. Es el fenómeno que el apunte de la cátedra ilustra con una línea transoceánica que queda sin usar el 98 % del tiempo, y la razón por la cual la opción Window Scale es hoy universal.

10.2. Producto Ancho de Banda-Retardo y Escalado de Ventana

Enunciado

Dos centros de cómputo están unidos por un enlace de 1 Gbps con RTT de 80 ms. Calcular el producto ancho de banda-retardo (BDP, Bandwidth-Delay Product), el factor mínimo de la opción Window Scale (RFC 7323) y el valor del campo Window Size.

Resolución

El BDP es la cantidad de datos que debe estar en vuelo para que el enlace nunca quede ocioso:

- $BDP = 10^9 \text{ bit/s} \times 0,08 \text{ s} = 8 \cdot 10^7 \text{ bit} = 10^7 \text{ bytes} = 10 \text{ MB}$.

Window Scale, negociada en los SYN, define un desplazamiento S (0 a 14) tal que la ventana efectiva es el campo $\times 2^S$. Se busca el menor S con $65.535 \times 2^S \geq 10.000.000$, es decir, $2^S \geq 152,6$:

- S = 7: $65.535 \times 128 = 8.388.480$ bytes, insuficiente.
- S = 8: $65.535 \times 256 = 16.776.960$ bytes, suficiente.
- Window Size = $10.000.000 / 256 = 39.062,5 \rightarrow$ se anuncia 39.063 (10.000.128 bytes).

Aplicando el mismo procedimiento a otros escenarios:

Enlace	RTT	BDP	S mínimo
100 Mbps	80 ms	1 MB	4
1 Gbps	10 ms	1,25 MB	5
1 Gbps	80 ms	10 MB	8
10 Gbps	80 ms	100 MB	11

Con S = 14 la ventana llega a $65.535 \times 16.384 = 1.073.725.440$ bytes, del orden de $2^{30} = 1 \text{ GB}$, el límite citado en el apunte. El BDP, y no la velocidad nominal, determina el tamaño de los buffers: a 10 Gbps y 80 ms el sistema debe reservar unos 100 MB por conexión. Si un middlebox elimina la opción del SYN, la conexión queda silenciosamente limitada a 64 KB.

10.3. Evolución de la Ventana de Congestión

Enunciado

Una conexión TCP con tamaño máximo de segmento (MSS, Maximum Segment Size) de 1 KB arranca con ventana de congestión (cwnd) de 1 MSS y umbral (sssthresh) de 32 KB. En la ronda 8 se produce una pérdida. Trazar cwnd ronda por ronda: (a) si la pérdida se detecta por timeout; (b) comparando Tahoe y Reno si se detecta por tres ACK duplicados; (c) cuantificar los datos enviados entre las rondas 9 y 16.

Resolución

Se adopta la convención de los ejercicios de Tanenbaum: cada ronda dura un RTT; en arranque lento (SS, slow start) la ventana se duplica por ronda sin superar sssthresh; en prevención de congestión (CA, congestion avoidance) crece 1 MSS por ronda.

La ventana crece 1, 2, 4, 8, 16 y 32 KB; en la ronda 6 alcanza el umbral y pasa a crecimiento lineal: 33 y 34 KB. Ante la pérdida, según RFC 5681, $sssthresh = 34 / 2 = 17$ KB. Ante un timeout, Tahoe y Reno llevan cwnd a 1 MSS. Ante tres ACK duplicados, Tahoe hace lo mismo, mientras que Reno aplica recuperación rápida (fast recovery) y continúa en CA con cwnd = 17 KB. La columna de Tahoe vale para el caso (a) con cualquier variante:

Ronda	cwnd Tahoe (KB)	Fase Tahoe	cwnd Reno (KB)	Fase Reno
1–5	1, 2, 4, 8, 16	SS	1, 2, 4, 8, 16	SS
6	32	Llega a sssthresh	32	Llega a sssthresh
7	33	CA	33	CA
8	34	CA, pérdida	34	CA, pérdida
9	1	SS (sssthresh 17)	17	CA
10	2	SS	18	CA
11	4	SS	19	CA
12	8	SS	20	CA
13	16	SS	21	CA
14	17	Llega a sssthresh	22	CA
15	18	CA	23	CA
16	19	CA	24	CA

Datos enviados entre las rondas 9 y 16:

- Tahoe: $1 + 2 + 4 + 8 + 16 + 17 + 18 + 19 = 85$ KB.
- Reno: $17 + 18 + \dots + 24 = 164$ KB, es decir, $164 / 85 \approx 1,93$ veces más.

Tres ACK duplicados indican que los segmentos posteriores al perdido siguen llegando, por lo que no tiene sentido reiniciar desde un segmento. Tahoe desperdicia cinco rondas en reconstruir la ventana; Reno solo aplica la reducción multiplicativa de AIMD (Additive Increase, Multiplicative Decrease) y sigue adelante, lo que explica que desplazara rápidamente a Tahoe.

10.4. Cálculo del Temporizador de Retransmisión (RTO)

Enunciado

Una conexión TCP obtiene las muestras de RTT 100, 120, 80 y 200 ms. Calcular tras cada una el RTT suavizado (SRTT), la variación (RTTVAR) y el temporizador de retransmisión (RTO, Retransmission Timeout) según Jacobson (RFC 6298), con $\alpha = 1/8$, $\beta = 1/4$ y $RTO = SRTT + 4 \cdot RTTVAR$.

Resolución

Con la primera muestra R: $SRTT = R$ y $RTTVAR = R / 2$. Con cada muestra R' posterior se actualiza primero RTTVAR, con el SRTT anterior:

- $RTTVAR = 0,75 \cdot RTTVAR + 0,25 \cdot |SRTT - R'|$; $SRTT = 0,875 \cdot SRTT + 0,125 \cdot R'$.

Cálculos:

- Muestra 1: $SRTT = 100$; $RTTVAR = 50$; $RTO = 100 + 200 = 300$ ms.
- Muestra 2: $RTTVAR = 37,5 + 0,25 \cdot 20 = 42,5$; $SRTT = 87,5 + 15 = 102,5$; $RTO = 102,5 + 170 = 272,5$ ms.
- Muestra 3: $RTTVAR = 31,875 + 0,25 \cdot 22,5 = 37,5$; $SRTT = 89,6875 + 10 = 99,6875$; $RTO = 99,6875 + 150 \approx 249,7$ ms.
- Muestra 4: $RTTVAR = 28,125 + 0,25 \cdot 100,3125 \approx 53,2$; $SRTT = 87,227 + 25 \approx 112,2$; $RTO = 112,227 + 212,813 \approx 325,0$ ms.

Muestra	R (ms)	RTTVAR (ms)	SRTT (ms)	RTO (ms)
1	100	50,0	100,0	300,0
2	120	42,5	102,5	272,5

Muestra	R (ms)	RTTVAR (ms)	SRTT (ms)	RTO (ms)
3	80	37,5	99,7	249,7
4	200	53,2	112,2	325,0

Si el segmento de la cuarta muestra hubiera sido retransmitido, el algoritmo de Karn indica descartarla —no se sabe a qué envío corresponde el ACK— y duplicar el RTO ante cada expiración: $249,7 \rightarrow 499,4 \rightarrow 998,8$ ms.

El SRTT se mueve lentamente —una muestra del doble de lo habitual lo lleva solo de 99,7 a 112,2 ms—, mientras que el término de variación reacciona rápido y es el que más eleva el RTO, evitando una retransmisión innecesaria. RFC 6298 recomienda además un RTO mínimo de 1 s (Linux usa 200 ms), piso que solo opera en redes de RTT bajo.

10.5. Eficiencia y Overhead de Encabezados

Enunciado

Comparar la eficiencia —bytes de aplicación sobre bytes que ocupan el medio— sobre Ethernet de: (a) una transferencia TCP/IPv4 con MSS 1460 bytes; (b) una llamada de voz sobre IP (VoIP) con códec G.711 que envía cada 20 ms 160 bytes sobre RTP (Real-time Transport Protocol), UDP (User Datagram Protocol) e IPv4. Calcular el ancho de banda real de la llamada.

Resolución

Encabezados: TCP 20 bytes, UDP 8, RTP 12, IPv4 20. Ethernet agrega 18 bytes (encabezado y FCS) y, en el medio, 20 más de preámbulo y separación entre tramas.

- TCP: paquete IP = $1460 + 40 = 1500$ bytes; trama = 1518; medio = 1538. Eficiencia = $1460 / 1538 \approx 94,9 \%$ ($1460 / 1518 \approx 96,2 \%$ sobre la trama).
- VoIP: $64.000 \text{ bit/s} \times 0,02 \text{ s} / 8 = 160$ bytes; paquete IP = $160 + 12 + 8 + 20 = 200$ bytes; trama = 218; medio = 238. Eficiencia = $160 / 238 \approx 67,2 \%$ ($160 / 218 \approx 73,4 \%$ sobre la trama).
- Ancho de banda de la llamada, con 50 paquetes por segundo: $200 \times 8 \times 50 = 80$ kbps a nivel IP y $238 \times 8 \times 50 = 95,2$ kbps en el medio.

Concepto	TCP masivo	VoIP sobre UDP/RTP
Carga útil	1460 B	160 B
Encabezados de capas 3 y 4	40 B	40 B
Ocupación del medio	1538 B	238 B
Eficiencia en el medio	94,9 %	67,2 %

Aunque el encabezado UDP es menor que el de TCP, la voz resulta mucho menos eficiente porque 40 bytes de encabezados pesan sobre una carga de 160: una llamada de 64 kbps ocupa casi 95 kbps. Con G.729 (20 bytes cada 20 ms) la eficiencia a nivel IP cae al 33 %, lo que justifica la compresión de encabezados. UDP se elige en tiempo real por la ausencia de retransmisiones y de bloqueo, no por el tamaño de su encabezado.

10.6. Agotamiento del Espacio de Números de Secuencia

Enunciado

El número de secuencia de TCP tiene 32 bits y numera bytes. Calcular cuánto tarda en recorrerse a 1, 10 y 100 Gbps, compararlo con un tiempo máximo de vida del segmento (MSL, Maximum Segment Lifetime) de 120 s y justificar el mecanismo PAWS (Protection Against Wrapped Sequence numbers).

Resolución

El espacio es de 2^{32} bytes = 4.294.967.296 bytes = 34.359.738.368 bit. Dividiendo por la tasa:

Velocidad	Vuelta completa	Vueltas por MSL de 120 s
100 Mbps	343,6 s	menos de una: seguro
1 Gbps	34,4 s	3,5
10 Gbps	3,44 s	35
100 Gbps	0,344 s	349

La velocidad crítica, a partir de la cual el espacio se recorre en menos de un MSL, es $34.359.738.368 \text{ bit} / 120 \text{ s} \approx 286,3 \text{ Mbps}$. Por encima de ella, un segmento retrasado de la vuelta anterior puede caer dentro de la ventana y ser aceptado.

PAWS (RFC 7323) usa la opción Timestamp: cada segmento lleva una marca de tiempo creciente de 32 bits y el receptor descarta los que traen una marca anterior a la última recibida, aunque su secuencia sea válida. Con un tic de 1 ms, la marca recién se agota en 2^{31} ms $\approx$ 24,9 días.

El supuesto de 1981 de que el espacio de secuencia se recorría en mucho más tiempo que la vida de un segmento dejó de valer a partir de unos 286 Mbps. Sin PAWS, un único segmento retrasado podría corromper silenciosamente los datos, porque el checksum verifica la integridad pero no la pertenencia a la vuelta correcta.

11. Seguridad en la Capa de Transporte

11.1. Vulnerabilidades Históricas de TCP y UDP

TCP y UDP no incorporan autenticación de origen ni cifrado: una conexión se identifica por la cuádrupla de direcciones y puertos, y la única barrera contra un atacante es el conocimiento de los números de secuencia en uso.

- Predicción del número de secuencia inicial (ISN, Initial Sequence Number): en 1985 Robert T. Morris mostró que 4.2BSD generaba ISN predecibles, lo que permite completar un handshake falsificando la IP de un tercero sin ver las respuestas. El ataque de 1994 atribuido a Kevin Mitnick contra Tsutomu Shimomura combinó esta técnica con un SYN flood contra el host de confianza. Hoy RFC 6528 exige ISN derivados de una función hash con un secreto.
- Secuestro de sesión (session hijacking): un atacante que observa el tráfico conoce las secuencias vigentes e inyecta datos propios; contra él solo protegen el cifrado y la autenticación en capas superiores.
- Inyección de RST: un segmento RST con secuencia dentro de la ventana aborta una conexión ajena. En 2004 se mostró que era factible a ciegas contra sesiones BGP largas, y RFC 5961 respondió exigiendo coincidencia exacta y un «challenge ACK».
- UDP, sin handshake, no ofrece siquiera esa protección débil: un datagrama con origen falsificado es indistinguible de uno legítimo.

11.2. Ataques de Denegación de Servicio: SYN Flood y Amplificación UDP

El SYN flood explota que cada SYN obliga al servidor a reservar estado en SYN_RECEIVED esperando un ACK que, con origen falsificado, nunca llega; la cola se agota y se rechazan clientes legítimos. Tras el ataque contra el proveedor Panix en 1996, Daniel J. Bernstein propuso las SYN cookies, que codifican el estado en el ISN del SYN-ACK; RFC 4987 cataloga esta y otras mitigaciones.

En la amplificación UDP, el atacante envía consultas pequeñas a servidores públicos falsificando como origen la IP de la víctima, que recibe respuestas mucho mayores. Factores de referencia publicados por la agencia estadounidense de ciberseguridad:

Protocolo	Puerto UDP	Factor de amplificación
DNS	53	28 a 54
NTP (monlist)	123	≈ 557
Memcached	11211	10.000 a 51.000

En febrero de 2018, GitHub recibió por esta vía un ataque de alrededor de 1,35 Tbps. La raíz es la falsificación del origen, que el filtrado de ingreso (BCP 38) busca eliminar con adopción todavía incompleta. Los protocolos modernos sobre UDP lo tienen en cuenta: QUIC no envía más de tres veces lo recibido de una dirección no validada, y DTLS exige un intercambio de cookie.

11.3. Escaneo de Puertos y Superficie de Ataque

El escaneo de puertos determina qué servicios escuchan en un host; la herramienta de referencia es nmap. El escaneo SYN envía solo el SYN: un SYN-ACK indica puerto abierto (y se responde RST), un RST indica cerrado y el silencio sugiere un filtro. En UDP, un ICMP de puerto inalcanzable indica cerrado, pero el silencio es ambiguo. Cada puerto abierto es superficie de ataque, por lo que la primera defensa es no exponer servicios innecesarios. La segunda es el firewall con estado (stateful), que sigue la máquina de estados de cada conexión, admite solo las respuestas a conexiones iniciadas desde adentro y puede limitar la tasa de SYN entrantes.

11.4. TLS y DTLS: Seguridad sobre la Capa de Transporte

TLS (Transport Layer Security) opera sobre TCP y aporta confidencialidad, integridad y autenticación mediante certificados. TLS 1.3 (RFC 8446, 2018) completa el handshake en un RTT: el ClientHello ya incluye una clave efímera ECDHE y todo lo posterior viaja cifrado; eliminó el intercambio RSA estático, por lo que el secreto hacia adelante es obligatorio, y ofrece un modo 0-RTT para reanudaciones, vulnerable a repetición. DTLS adapta TLS a UDP —para WebRTC o VPN— con números de secuencia explícitos, retransmisión propia del handshake y cookies anti-amplificación; su versión vigente es DTLS 1.3 (RFC 9147). QUIC integra TLS 1.3 en el propio transporte y cifra también casi todo su encabezado, lo que elimina de raíz la inyección de RST y el secuestro de sesión.

12. La Capa de Transporte en la Práctica Profesional

12.1. Impacto en el Desarrollo de Software

Muchas fallas de rendimiento de los sistemas distribuidos provienen de decisiones de transporte tomadas por omisión:

- Timeouts y reintentos: toda operación de red debe tener timeout de conexión y de lectura, y los reintentos deben usar retroceso exponencial, por la misma razón por la que TCP duplica su RTO.
- Keepalive: en Linux la primera sonda se envía tras 2 horas de inactividad, mucho más de lo que muchos NAT conservan una entrada; las aplicaciones persistentes usan keepalives de minutos o latidos propios.
- Pools de conexiones: cada conexión nueva cuesta handshakes de TCP y TLS y varios RTT de arranque lento; reutilizarlas evita ese costo y la acumulación de sockets en TIME_WAIT.
- Nagle y TCP_NODELAY: Nagle, combinado con el ACK retardado, puede introducir demoras de decenas a cientos de milisegundos en protocolos de pedido y respuesta, por lo que las aplicaciones interactivas lo desactivan.
- Bloqueo de cabeza de línea: una pérdida en TCP detiene todos los flujos que HTTP/2 multiplexa sobre la conexión; por eso HTTP/3 usa QUIC.

- Elección del transporte: REST, gRPC y WebSockets (RFC 6455) se apoyan en TCP; los medios en tiempo real usan UDP con RTP o WebRTC; QUIC conviene cuando se necesitan varios flujos independientes y movilidad.

12.2. Herramientas de Diagnóstico: netstat, ss y Wireshark

El comando netstat lista conexiones con sus estados (netstat -ano en Windows agrega el proceso), y en Linux fue reemplazado por ss: ss -tan muestra los estados y ss -ti expone cwnd, ssthresh, RTT y RTO de cada conexión. Muchos sockets en SYN_RECV sugieren un SYN flood; muchos en TIME_WAIT, conexiones cortas no reutilizadas; y una acumulación de CLOSE_WAIT es casi siempre un error de programación: el otro extremo cerró, pero la aplicación nunca llamó a close.

Wireshark captura y decodifica los paquetes capa por capa. Un handshake típico, con números de secuencia relativos, se ve así:

No.	Origen	Destino	Info
1	192.168.1.10	203.0.113.5	51514 → 443 [SYN] Seq=0 Win=64240 MSS=1460 SACK_PERM TSval WS=256
2	203.0.113.5	192.168.1.10	443 → 51514 [SYN, ACK] Seq=0 Ack=1
3	192.168.1.10	203.0.113.5	51514 → 443 [ACK] Seq=1 Ack=1
4	192.168.1.10	203.0.113.5	Client Hello (TLS 1.3)

Se observa la confirmación del ISN más uno, la negociación de MSS, SACK y Window Scale en el SYN, y el inicio de TLS tras el tercer segmento; el tiempo entre SYN y SYN-ACK mide el RTT. Filtros como tcp.analysis.retransmission permiten aislar retransmisiones.

12.3. NAT, Middleboxes y la Osificación del Transporte

Entre los extremos operan middleboxes —NAT, firewalls, balanceadores, proxies— que inspeccionan y modifican el encabezado de transporte. Como solo reconocen TCP y UDP, protocolos como SCTP o DCCP son descartados por buena parte de ellos, y aun en TCP una fracción de los caminos elimina opciones desconocidas o reescribe secuencias. A esta osificación se suma que TCP vive en el kernel, por lo que cada mejora depende de actualizar miles de millones de dispositivos. QUIC responde encapsulándose sobre UDP en el puerto 443, que atraviesa casi todos los NAT; implementándose en espacio de usuario, para evolucionar con cada actualización del navegador; y cifrando su encabezado para que los middleboxes no puedan depender de sus campos.

12.4. Centros de Datos, Baja Latencia y Tendencias

En los centros de datos, con RTT de microsegundos y ráfagas que saturan los conmutadores, DCTCP (RFC 8257) usa marcas ECN tempranas y reduce $cwnd$ en proporción a la fracción α de paquetes marcados, según $cwnd \leftarrow cwnd \times (1 - \alpha/2)$, manteniendo colas cortas. BBR, de Google (2016), abandona la pérdida como señal: estima el ancho de banda del cuello de botella y el RTT mínimo para mantener en vuelo cerca de un BDP. L4S (RFC 9330 a 9332, 2023) extiende a Internet colas casi vacías para tráfico interactivo mediante ECN, y RDMA delega el transporte a la placa de red en entornos de alto rendimiento. La tendencia es que la latencia de cola importe tanto como el throughput y que la señalización explícita reemplace a la pérdida.

12.5. Conectividad en Argentina y su Relación con el Transporte

La conectividad internacional de Argentina depende de cables submarinos con amarre principal en Las Toninas, y el RTT hacia la costa este de Estados Unidos se ubica típicamente entre 120 y 160 ms, superando los 200 ms hacia Europa. Ese RTT alarga el arranque lento y la recuperación tras pérdidas: con una aproximación clásica del throughput de TCP en régimen, $(MSS / RTT) \times 1,22 / \sqrt{p}$, con MSS de 1460 bytes y pérdida del 0,01 %, se obtienen unos 9,5 Mbps a 150 ms frente a unos 95 Mbps a 15 ms. Las CDN con cachés dentro de los proveedores locales y los puntos de intercambio de CABASE —Rosario entre ellos— reducen el RTT a pocos milisegundos para el contenido popular, aunque el problema persiste para servicios alojados solo en el exterior. Las redes móviles agregan latencia variable y pérdidas no causadas por congestión. Por eso el handshake de un RTT de QUIC, la migración de conexiones y algoritmos como BBR benefician especialmente a usuarios lejanos de los servidores, y HTTP/3, soportado por los navegadores mayoritarios y las principales CDN, les llega sin inversión local.

13. Conclusión

A lo largo de este trabajo se analizó la capa de transporte desde sus fundamentos conceptuales hasta sus implementaciones actuales, recorriendo los servicios y primitivas que ofrece a las aplicaciones, los problemas que debe resolver para

comunicar procesos de extremo a extremo y los protocolos —UDP, TCP y sus sucesores— que dan respuesta a esos problemas en la Internet real.

La capa de transporte es la primera que opera de extremo a extremo, por encima de una capa de red que no garantiza entrega, orden ni tiempo, y su función es transformar ese servicio de mejor esfuerzo en lo que cada aplicación necesita. UDP es la opción mínima: multiplexación por puertos y verificación de integridad, con todo lo demás en manos de la aplicación. TCP es la opción completa: un flujo de bytes confiable y ordenado, construido con números de secuencia, confirmaciones, retransmisiones, ventanas deslizantes y una máquina de estados que resuelve problemas aparentemente simples, como abrir y cerrar una conexión en presencia de duplicados retrasados y mensajes perdidos.

El aporte más significativo de TCP no es, sin embargo, la confiabilidad, sino el control de congestión. Tras los colapsos de mediados de la década de 1980, el trabajo de Jacobson y el análisis de Chiu y Jain mostraron que una red compartida podía estabilizarse sin coordinación central, con una regla aplicada por cada emisor: aumentar de a poco y reducir a la mitad ante la congestión. Los ejercicios del informe mostraron cómo esa regla gobierna la ventana, cómo se estima el temporizador de retransmisión y por qué fueron necesarias la recuperación rápida, la confirmación selectiva, el escalado de ventana y las marcas de tiempo a medida que las redes crecieron en velocidad y distancia.

Esa evolución deja una lección de diseño. TCP, concebido para kilobits por segundo, sigue funcionando a cientos de gigabits gracias a sus opciones extensibles y a la separación entre protocolo y algoritmo de congestión, pero acumuló límites difíciles de superar: el bloqueo de cabeza de línea, los handshakes sucesivos de transporte y seguridad, la falta de cifrado nativo y la osificación causada por los dispositivos intermedios. QUIC, SCTP y MPTCP responden a esos límites, y el éxito de QUIC muestra que la viabilidad de un protocolo depende tanto de su diseño como de su capacidad de desplegarse.

Para los ingenieros en sistemas, estos conocimientos tienen consecuencias inmediatas: elegir entre TCP, UDP o QUIC, configurar timeouts y keepalives, decidir si desactivar Nagle, dimensionar buffers según el producto ancho de banda-retardo o interpretar una captura de Wireshark son tareas habituales. A ello se

suma la seguridad: la falta de autenticación en TCP y UDP explica ataques que van de la predicción de secuencias a la amplificación, y justifica que TLS, DTLS y el cifrado de QUIC sean hoy parte inseparable del transporte.

En última instancia, decisiones tomadas hace más de cuarenta años siguen condicionando la experiencia de miles de millones de usuarios, y la evolución reciente apunta a una Internet en la que la latencia y su previsibilidad importan tanto como el throughput. Para un país alejado de los grandes centros de contenido como Argentina, cada RTT ahorrado por un protocolo mejor diseñado se traduce directamente en aplicaciones más rápidas.

14. Bibliografía

Vitri, H., Baró, G. y Travaglino, E. (s. f.). Capa de Transporte — Servicios y Primitivas. Apunte de cátedra, Redes de Información. UTN Facultad Regional Rosario.

Vitri, H., Baró, G. y Travaglino, E. (s. f.). Características de Transporte. Apunte de cátedra, Redes de Información. UTN Facultad Regional Rosario.

Vitri, H., Baró, G. y Travaglino, E. (s. f.). Protocolo TCP. Apunte de cátedra, Redes de Información. UTN Facultad Regional Rosario.

Tanenbaum, A. S. y Wetherall, D. J. (2011). Computer Networks (5th ed.). Pearson Prentice Hall.

Tanenbaum, A. S., Feamster, N. y Wetherall, D. J. (2021). Computer Networks (6th ed.). Pearson.

Kurose, J. F. y Ross, K. W. (2021). Computer Networking: A Top-Down Approach (8th ed.). Pearson.

Forouzan, B. A. (2013). Data Communications and Networking (5th ed.). McGraw-Hill.

Fall, K. R. y Stevens, W. R. (2011). TCP/IP Illustrated, Volume 1: The Protocols (2nd ed.). Addison-Wesley.

Postel, J. (1980). User Datagram Protocol (RFC 768). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc768>

Postel, J. (1981). Transmission Control Protocol (RFC 793). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc793>

Eddy, W. (Ed.). (2022). Transmission Control Protocol (TCP) (RFC 9293). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc9293>

Allman, M., Paxson, V. y Blanton, E. (2009). TCP Congestion Control (RFC 5681). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc5681>

Paxson, V., Allman, M., Chu, J. y Sargent, M. (2011). Computing TCP's Retransmission Timer (RFC 6298). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc6298>

Borman, D., Braden, B., Jacobson, V. y Scheffenegger, R. (Ed.). (2014). TCP Extensions for High Performance (RFC 7323). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc7323>

Iyengar, J. y Thomson, M. (Eds.). (2021). QUIC: A UDP-Based Multiplexed and Secure Transport (RFC 9000). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc9000>

Eddy, W. (2007). TCP SYN Flooding Attacks and Common Mitigations (RFC 4987). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc4987>

Rescorla, E. (2018). The Transport Layer Security (TLS) Protocol Version 1.3 (RFC 8446). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc8446>

Jacobson, V. (1988). Congestion Avoidance and Control. ACM SIGCOMM Computer Communication Review, 18(4), 314–329.

Chiu, D.-M. y Jain, R. (1989). Analysis of the Increase and Decrease Algorithms for Congestion Avoidance in Computer Networks. Computer Networks and ISDN Systems, 17(1), 1–14.

Ha, S., Rhee, I. y Xu, L. (2008). CUBIC: A New TCP-Friendly High-Speed TCP Variant. ACM SIGOPS Operating Systems Review, 42(5), 64–74.

Cardwell, N., Cheng, Y., Gunn, C. S., Hassas Yeganeh, S. y Jacobson, V. (2016). BBR: Congestion-Based Congestion Control. *ACM Queue*, 14(5), 20–53.

15. Anexo: Autoevaluación — Preguntas y Respuestas

Pregunta 1: ¿Qué responsabilidades tiene la capa de transporte?

Respuesta: La capa de transporte brinda comunicación extremo a extremo, confiable y eficiente, entre procesos de aplicación ubicados en distintos hosts, por encima de una capa de red que solo ofrece un servicio de mejor esfuerzo. Sus funciones son: dividir los datos de la aplicación en segmentos y reensamblarlos en el destino; multiplexar y demultiplexar las conversaciones de distintas aplicaciones mediante puertos; detectar errores mediante el checksum; y, en el caso de TCP, garantizar la entrega ordenada y confiable mediante confirmaciones y retransmisiones, regular el flujo según la capacidad del receptor y controlar la congestión de la red.

Pregunta 2: ¿Cómo controla la congestión el protocolo UDP?

Respuesta: UDP no controla la congestión. Tampoco implementa control de flujo: envía los datagramas al ritmo que le impone la aplicación, sin adaptarse al estado de la red ni a la capacidad del receptor, por lo que un emisor UDP puede contribuir a saturar los enlaces. Si se necesita control de congestión, debe implementarlo la propia aplicación o un protocolo construido sobre UDP; QUIC, por ejemplo, incorpora mecanismos equivalentes a los de TCP, y DCCP fue diseñado para ofrecer control de congestión sin confiabilidad.

Pregunta 3: ¿Qué sucede si UDP detecta un error? ¿Y TCP?

Respuesta: Si UDP detecta mediante el checksum que un datagrama llegó corrupto, simplemente lo descarta, sin notificar al emisor ni intentar recuperarlo; la aplicación nunca lo recibe. TCP también descarta el segmento corrupto, pero como no lo confirma, el emisor lo retransmite al vencer el temporizador o al recibir ACK duplicados. De ese modo TCP garantiza que la aplicación reciba todos los bytes, sin errores y en orden.

Pregunta 4: ¿Cómo se grafica un ejemplo de conexión mediante el three-way handshake?

Respuesta: Se dibujan dos líneas de tiempo verticales, una para el cliente y otra para el servidor, que previamente ejecutó LISTEN. Primera flecha, del cliente al servidor: segmento con SYN = 1, ACK = 0 y número de secuencia x, que incluye el puerto destino y el MSS. Segunda flecha, del servidor al cliente: segmento SYN-ACK con SYN = 1, ACK = 1, número de secuencia y y acknowledgement x + 1. Tercera flecha, del cliente al servidor: segmento con ACK = 1, número de secuencia x + 1 y acknowledgement y + 1. A partir de ese momento ambos extremos quedan en estado ESTABLISHED y el flujo de datos es bidireccional.

Pregunta 5: ¿Cuáles son las diferencias entre TCP y UDP?

Respuesta: TCP es orientado a conexión, con establecimiento mediante three-way handshake, mientras que UDP no tiene conexión y envía de inmediato. TCP garantiza entrega, orden y ausencia de duplicados mediante confirmaciones y retransmisiones; UDP no garantiza nada de eso. TCP realiza control de flujo y de congestión; UDP no. El encabezado de TCP ocupa 20 bytes como mínimo y el de UDP 8 bytes. TCP ofrece un flujo de bytes continuo, mientras que UDP preserva los límites de cada mensaje. TCP se usa en la web, el correo y la transferencia de archivos; UDP en voz y video en tiempo real, DNS, DHCP y multicast.

Pregunta 6: ¿Qué es un TSAP y cómo se organizan los puertos?

Respuesta: El TSAP (Transport Service Access Point) identifica a un proceso dentro de un host y en Internet corresponde al puerto, de 16 bits. Los menores a 1024 están reservados para servicios conocidos, como 21 FTP, 25 SMTP u 80 HTTP. Una conexión se identifica por la cuádrupla de IP y puerto de origen y destino.

Pregunta 7: ¿En qué consiste el problema de los dos ejércitos y qué implica para la liberación de conexiones?

Respuesta: Dos ejércitos que se comunican por mensajeros capturables nunca pueden estar seguros de que el último mensaje llegó, por lo que no existe un protocolo finito que garantice el acuerdo. En consecuencia, TCP libera la conexión con FIN y ACK en cada sentido y temporizadores: si un mensaje se pierde, cada extremo cierra por su cuenta.

Pregunta 8: ¿Qué son los duplicados retrasados y cómo se evita que se confundan con segmentos nuevos?

Respuesta: Son segmentos demorados en la red que llegan después de haber sido retransmitidos, y podrían repetir una operación. Se evitan limitando la vida de los segmentos (unos 120 s) y con el three-way handshake de Tomlinson, en el que cada extremo elige un número de secuencia inicial, aleatorio en TCP, que el otro debe confirmar.

Pregunta 9: ¿Qué es AIMD y por qué converge a una asignación justa?

Respuesta: AIMD aumenta la tasa en una cantidad constante sin congestión y la reduce a la mitad al detectarla. Chiu y Jain demostraron que el aumento aditivo conserva las diferencias entre flujos y la reducción multiplicativa las achica, por lo que las tasas convergen a un reparto eficiente y equitativo.

Pregunta 10: ¿Cómo funciona el arranque lento (slow start) de TCP?

Respuesta: La ventana de congestión arranca pequeña y crece un MSS por cada ACK, duplicándose en cada RTT. Termina al alcanzar ssthresh, donde el crecimiento pasa a ser lineal, o ante una pérdida, que fija el umbral en la mitad de la ventana.

Pregunta 11: ¿Qué diferencia hay entre TCP Tahoe y TCP Reno?

Respuesta: Ante tres ACK duplicados, Tahoe reinicia la ventana en un segmento y vuelve al arranque lento, mientras que Reno aplica recuperación rápida: reduce la ventana a la mitad y sigue en crecimiento lineal. Ante un timeout ambas se comportan igual.

Pregunta 12: ¿Qué es SACK y qué ventaja aporta?

Respuesta: SACK (Selective Acknowledgement) permite al receptor informar rangos de bytes recibidos fuera de orden, de modo que el emisor retransmite solo lo que falta. Si se pierden los segmentos 2 y 5, se envía ACK del 1 y SACK de 3, 4 y 6.

Pregunta 13: ¿Qué es el algoritmo de Nagle y cuándo conviene desactivarlo?

Respuesta: Nagle retiene los datos pequeños mientras haya un segmento sin confirmar, para evitar muchos segmentos diminutos. Combinado con el ACK retardado puede agregar demoras notables, por lo que las aplicaciones interactivas lo desactivan con TCP_NODELAY.

Pregunta 14: ¿Cómo calcula TCP el temporizador de retransmisión y qué establece el algoritmo de Karn?

Respuesta: Jacobson mantiene $SRTT = 0,875 \cdot SRTT + 0,125 \cdot R$ y $RTTVAR = 0,75 \cdot RTTVAR + 0,25 \cdot |SRTT - R|$, y fija $RTO = SRTT + 4 \cdot RTTVAR$. Karn establece no medir RTT sobre segmentos retransmitidos y duplicar el RTO ante cada timeout.

Pregunta 15: ¿Qué es el estado TIME_WAIT y por qué existe?

Respuesta: Es el estado del extremo que cierra activamente, que espera dos veces el MSL tras el último ACK. Permite responder un FIN retransmitido si ese ACK se perdió y asegura que los segmentos viejos expiren antes de reutilizar la misma cuádrupla.

Pregunta 16: ¿En qué consiste un ataque SYN flood y cómo lo mitigan las SYN cookies?

Respuesta: El atacante envía muchos SYN con origen falsificado sin completar el handshake, agotando la cola de conexiones semiabiertas. Con SYN cookies el servidor no guarda estado: lo codifica en el ISN del SYN-ACK y reconstruye la conexión solo al recibir un ACK válido.

Pregunta 17: ¿Qué es QUIC y por qué se construyó sobre UDP?

Respuesta: QUIC (RFC 9000) es un transporte confiable, multiplexado y cifrado que integra TLS 1.3 y sirve de base a HTTP/3, sin bloqueo de cabeza de línea entre flujos. Usa UDP porque los NAT y firewalls descartan transportes desconocidos y para poder implementarse en espacio de usuario.

Pregunta 18: ¿Qué características distinguen a SCTP de TCP?

Respuesta: SCTP preserva los límites de los mensajes, admite múltiples flujos independientes por asociación y soporta multihoming con varias direcciones IP por extremo. Su handshake de cuatro vías con cookie lo hace inmune al SYN flood, aunque los middleboxes limitan su despliegue en Internet.

Pregunta 19: ¿Qué es el producto ancho de banda-retardo y por qué hace falta la opción Window Scale?

Respuesta: Es la cantidad de datos que debe estar en vuelo para mantener ocupado un enlace: capacidad por RTT, por ejemplo 10 MB para 1 Gbps y 80 ms.

Como el campo Window Size limita la ventana a 65.535 bytes, Window Scale lo multiplica por 2^S , con S hasta 14, hasta aproximadamente 1 GB.

Pregunta 20: ¿Qué indican los flags del encabezado TCP?

Respuesta: SYN sincroniza secuencias al abrir, ACK valida el campo de confirmación, FIN cierra un sentido, RST aborta la conexión, PSH pide entregar sin buffer y URG señala datos urgentes. ECE y CWR se usan con ECN: el primero avisa congestión y el segundo confirma la reducción de la ventana.